

	VELEUČILIŠTE U ŠIBENIKU	
	<i>Trg Andrije Hebranga 11, 22000 Šibenik, www.vus.hr</i>	

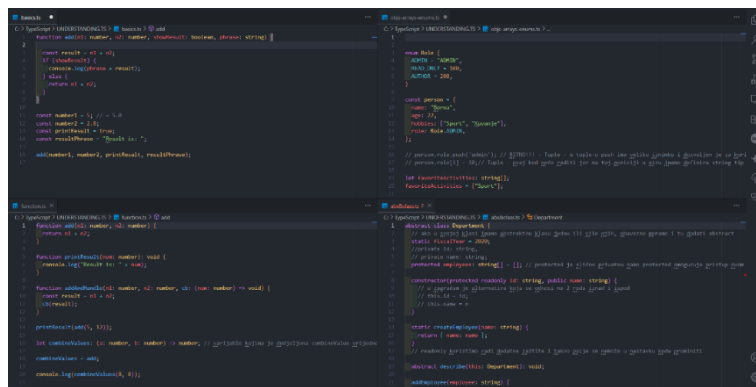
DNEVNIK STRUČNE PRAKSE

PODATCI O STUDIJU	
STUDIJ:	Stručni prijediplomski studij Poslovna informatika
AKADEMSKA GODINA:	2024./25.
NOSITELJ KOLEGIJA:	dr. sc. Ivan Livaja, v. pred.
PODATCI O STUDENTU/ICI	
PREZIME I IME:	Borna Mušćet
MATIČNI BROJ STUDENTA:	1219063634
KONTAKT E-MAIL:	bornamuscet@gmail.com
PODATCI O PRAVNOM SUBJEKTU OBAVLJANJA STRUČNE PRAKSE	
NAZIV:	COGNISM d.o.o.
ADRESA:	Trg kneza Višeslava 6, Zadar
MENTOR:	Stjepan Buljat
KONTAKT MENTORA:	Stjepan.buljat@cognism.com

OPIS OBAVLJENIH RADNIH ZADATAKA (VJEŽBI) NA PRAKSI

DAN 1.
02.10.2024.

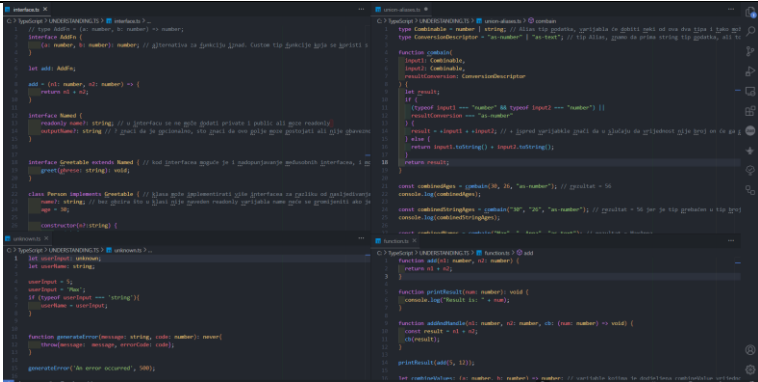
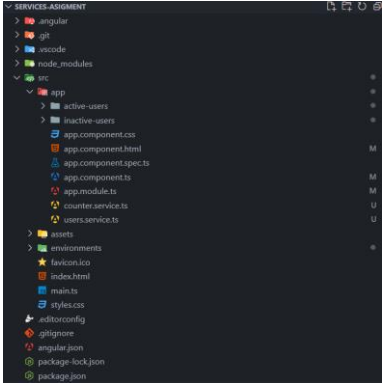
Prvi zadatak, prvog dana bio je upoznavanjem s JavaScript ES6. S mentorom je dogovoren prolazak kroz trening preko Udeemy platforme. Za početak smo se upoznavali sa JavaScriptom, usporedili smo koje su nove stvari dodane u odnosu na verziju ES5. S tim smo zaključili početni odjeljak baziran na upoznavanje sa JavaScripta ES6. Ostatak radnog dana iskoristili smo za trening razumijevanja TypeScript-a, u ovom dijelu prolazimo kroz teoretski ali i kroz praktični dio u kojem ćemo u nastavku imati par jednostavnih projekta koji će nam koristiti kao dodatna vježba. Krenuli smo sa treningom i za početak upoznajemo se što je i za što nam služi TypeScript, kako ga implementirati u svoj projekt i kako ga koristiti. Prolazimo kroz osnove TypeScripta, razliku između tipova JS-a i TS-a, tipove nizova, rad s Tuples i Enums, korištenje funkcije kao tipa, unknown i never tip itd. Nakon svakih nekoliko lekcija imamo i mali kviz koji nam služi da vidimo koliko smo razumjeli ono što smo prošli.



Prikaz TypeScript zadatka

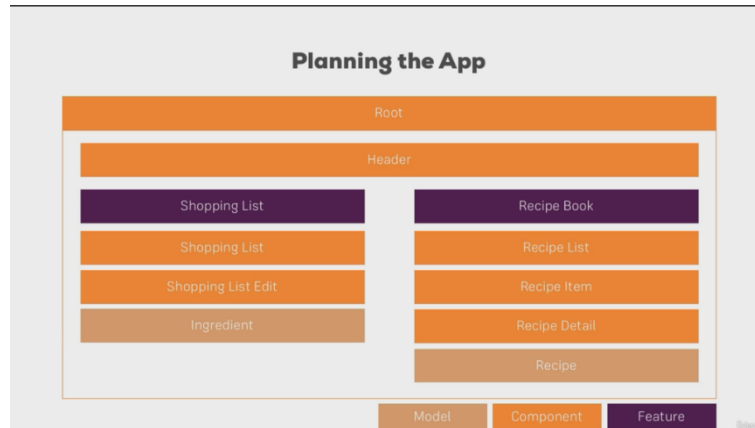
DAN 2.
03.10.2024.

Današnjim danom nastavljamo dalje sa treningom TypeScript-a, dolazimo do Klasa i Interface-a. Kroz poglavlje prošli smo kroz dva kviza i nakon toga došli smo do naprednijih tipova (Intersection, Guards, Casting, ...), sljedeće na redu su Generics poglavlje gdje smo se upoznali sa kreiranjem generics funkcija, naučili smo što su i zašto se koriste, te kako raditi sa njima. Zadnje poglavlje su Decorators koje sadržavaju kreiranje, rad, i ostale radnje vezane za upoznavanje i pripremu za daljnje korištenje, s tim završavamo trening upoznavanja s TypeScript-om te prelazimo na novi kompletan trening namijenjen Angular-u.

	 <i>Prikaz TypeScript zadatka</i>
DAN 3. 04.10.2024.	Trećim danom praktične nastave krećemo s kompletni vodičem kroz Angular trening, u ovom dijelu kao i prije imali smo teoretski kao i praktični dio koji će biti dosta napredniji od prethodnog. Za početak upoznajemo se što je Angular, zašto ga koristimo, kreiramo naš prvi Angular projekt i postavljamo sve potrebno kako bi nesmetano i točno radio. Nakon toga dolazimo do komponenti Angular-a kroz koje detaljno prolazimo kako bi se što bolje upoznali, prolazimo i kroz dio predložaka i servisa. Do kraja dana implementirali smo prvi praktični zadatak, radimo mali projekt koji koristi dva servisa, sadrži dvije liste za aktivne i neaktivne korisnike i klikom na gumb kraj imena korisnika postavljamo njegovu vrijednost na stanje u kojem je korisnik trenutno, tj. postavljamo korisnika aktivnog ili neaktivnog. Nakon kratkog projekta dolazimo do dijela rada sa Angular modulima (NgModule), kreiramo prvi modul i implementiramo Bootstrap.  <i>Prikaz strukture projekta</i>
DAN 4. 07.10.2024	Danas nastavljamo dalje i prolazimo kroz razumijevanje Error poruka i ispravljanje istih, te ulazimo u konkretno upoznavanje sa komponentama i predlošcima (razdvajanje komponente u više komponenti, vezanje svojstava, ograničavanje CSS stilova na komponente, razumijevanje i konfiguriranje View

	Encapsulation, razumijevanje signala efekata, TS i modeli tipova, detaljan prikaz predložka za petlje, komunikacija između komponenti i upravljanje stanjem itd.). Uz to prolazimo kroz poglavlja poboljšavanja elemenata pomoću direktiva (razumijevanje direktiva, ngModel, ngIf, direktive i ubrizgavanje ovisnosti itd) i kroz transformiranje vrijednosti pomoću Pipes. Sve u svemu dan baziran na praćenje video materijala i vlastito istraživanje.
DAN 5. 09.10.2024.	Petim danom stručne prakse fokus je na poglavljima u kojima ulazimo u dubinu razumijevanja servisa i dependency injection-sa, koristimo Angular-ov mehanizam za ubrizgavanje ovisnosti kao i pružanje usluge putem element injektora, analiziramo ubrizgavanje ovisnosti pomoću Angular DevTools-a. Nakon kratke pauze dolazimo do dijela razumijevanja detekcije promjena i kako ih Angular detektira, koristimo strategiju onPush te signale za dijeljenje podataka između komponenti. Počinjemo i sa radom s RxJS (Observables) Polako se približavamo malo većem projektu koji ćemo odraditi vezano za Angular trening ali prije toga ostatak dana predali smo radu na upoznavanju sa slanjima HTTP zahtjeva i rukovanje odgovorima, prolazimo kroz primjere pružanja HttpClient-a prilikom korištenja NgModule-a, slanje GET zahtjeva za dohvaćanje podataka, transformiranje i korištenje podataka odgovora, brisanje poslanih zahtjeva, itd.. S tim završavamo naš dan!
DAN 6. 10.10.2024	Fokus današnjeg dana kreće s odjeljkom koji pokriva sve aspekte rada s obrascima, od jednostavnih unosa korisničkih podataka do složenih reaktivnih obrazaca s prilagođenim validacijama. Modul započinje uvodom u obrasce vođene predloškom i reaktivne obrasce, gdje se objašnjavaju razlike između ta dva pristupa. Sljedeći odjeljak bavi se temeljima usmjeravanja u Angular aplikacijama te kako izgraditi jednostranične aplikacije s višestrukim stranicama putem usmjeravanja. Zapčinje s uvodom u osnovne pojmove o usmjeravanju i postupno vodi do naprednijih koncepata poput dinamičkih ruta, gniježđenih ruta, te rada s parametrima rute. Prije projekta o kojem smo ranije pričali imamo još jedan odjeljak a to je 'Razdvajanje koda i odgodivi prikazi'. Ovaj odjeljak objašnjava tehnike razdvajanja koda i kako implementirati "lazy loading" u Angular aplikacijama kako bi se optimizirala brzina učitavanja.

Dan smo zaključili zadatkom kojim smo postavili vizualnu strukturu projekta koji će nas pratiti kroz ovaj trening

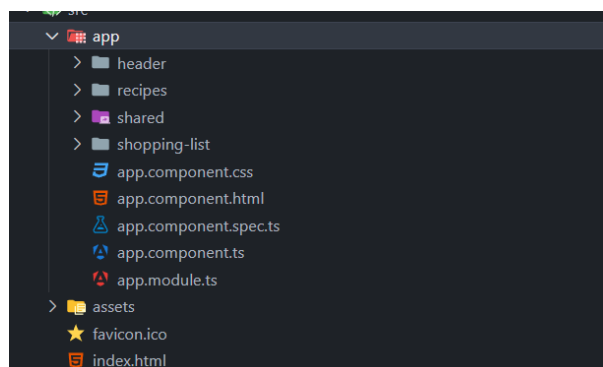


Vizualni prikaz strukture projekta

DAN 7.
11.10.2024

Stigli smo i do malo konkretnijeg zadatka u vezi projekta, današnji dio rezerviran je za odjeljak koji pokriva osnovne koncepte Angular-a, kao i ključne komponente neophodne za razvoj aplikacija. Upoznajemo se s načinom na koji Angular aplikacija započinje rad, procesom kreiranja novih komponenti, te s važnim konceptima poput interpolacije niza, vezanja svojstava, događaja i dvostrukog vezivanja podataka.

Ostatak dana iskoristit ćemo za praktičnu primjenu znanja kroz izradu projekta koristeći osnovne Angular koncepte. Prolazimo kroz cjelokupan proces od planiranja aplikacije do implementacije funkcionalnosti i prikaza podataka.

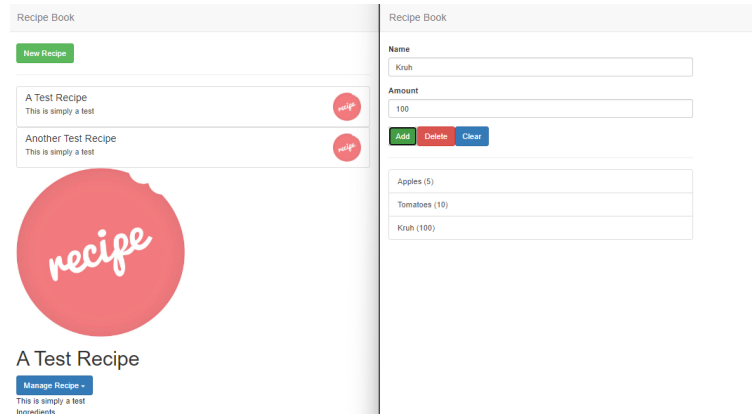


Prikaz strukture projekta

	Recipe Book RecipesShopping ListManage New Recipe A Test Recipe This is simply a test recipe A Test Recipe This is simply a test recipe recipe-item works! Recipe Name Manage Recipe - Description Ingredients Name Amount Add Delete Clear Apples (5) Tomatoes (10) Recipe Book projekt
DAN 8. 14.10.2024	Zadatak osmog dana bio je baziran na učenju kako razdijeliti aplikacije na komponente, što mi je pomoglo bolje organizirati kod i omogućilo jednostavnije održavanje aplikacije. Posebno korisno bilo je vezivanje na prilagođena svojstva i događaje unutar komponenti, kao i korištenje aliasa za prilagođena svojstva i događaje. Drugi dio dana posvetili smo životnom ciklusu komponente, gdje smo pratili faze poput ngOnInit i ngOnDestroy, koje upravljaju ponašanjem aplikacije tijekom njenog rada. Kao završnu vježbu, odradili sam zadatak koji je kombinirao vezivanje podataka i inkapsulaciju pogleda, čime smo stekli praktično iskustvo u primjeni ovih koncepata. Add new Servers or blueprint! Server Name Test Server Content Testrange Add Server Add Server Blueprint Change first Element Destroy first Component! Changed! Testrange Test Testrange Test Testrange Test Testrange 'Server' projekt

DAN 9.
15.10.2024

Današnji zadatak bio je implementiranje funkcionalnost jučerašnjeg projekta u naš glavni projekt. Radimo na procesu kako prosljeđivati podatke o receptima putem vezivanja svojstava te smo s tim omogućili korisnicima dodavanje sastojaka na popis za kupovinu, što je funkcionalnost koja direktno koristi mehanizme vezivanja podataka.

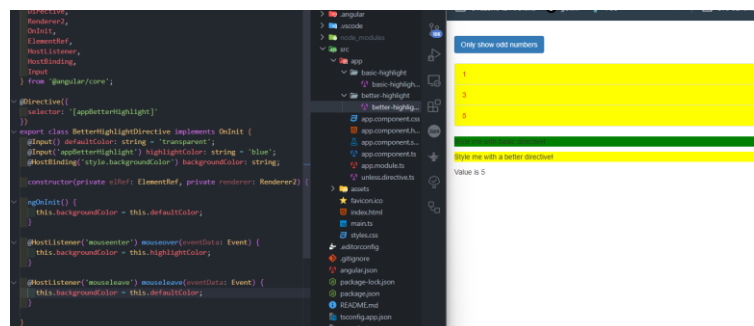


Nastavak rada na glavnom projektu

Nakon implementacije, ostatak dana iskoristit ćemo za upoznavanje sa Angular direktivama. Naučili sam kako kreirati direktive koje mijenjaju ponašanje elemenata u aplikaciji.

Također sam prošli kroz proces kako koristiti HostListener za slušanje događaja na host elementima, te HostBinding za vezivanje svojstava na host elemente, čime je postignuta puna kontrola nad ponašanjem komponenti.

Kreirali smo i jednostavan projekt koji nam je dodatno razjasnio kako i na koji način rukovati s ovim Angular funkcionalnostima.



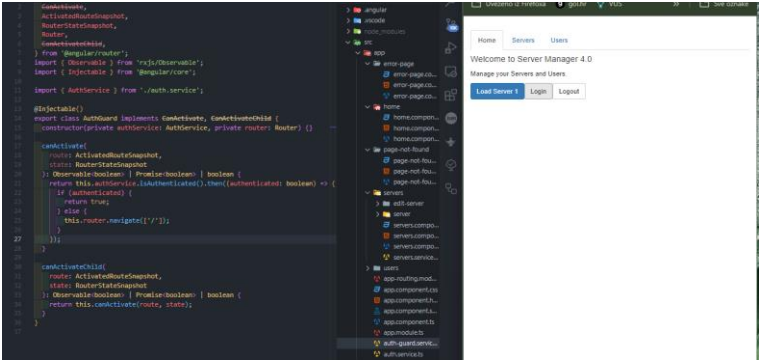
Projekt vezan za poglavlje Angular direktiva

DAN 10.
16.10.2024

Dvoznamenkastim brojem dana pohađanja praktične nastave započeli sam s upoznavanjem o važnosti usluga u Angular aplikacijama i zašto su one potrebne za komunikaciju i dijeljenje podataka među komponentama.

Zadatak je bio naučiti kako kreirati uslugu za evidentiranje (Logging Service) i kako je ubrizgati u različite komponente. Također smo prikazali i alternativnu sintaksu ubrizgavanja, koja omogućava fleksibilnost u načinu na koji se usluge koriste.

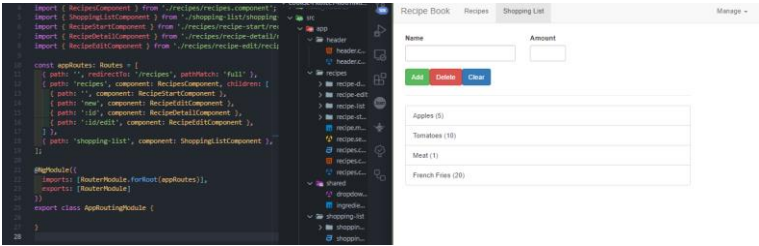
Sljedeće na redu je promjena stranica pomoću usmjeravanja (Routing). Usmjerivač (Router) omogućava jednostavno prebacivanje između različitih stranica bez potrebe za ponovnim učitavanjem cijele aplikacije.



Prikaz 'Routing' projekta

Sad kada smo se upoznali sa usmjeravanjem (Routing) u Angularu, trebamo to implementirati u naš 'Shopping and Recipe Book' projekt. Prvi zadatak bio je usmjeriti se na postavljanje ruta, zatim smo dodali navigaciju u aplikaciju, što omogućava korisnicima lako prebacivanje između različitih stranica.

Poseban fokus bio je na konfiguriranju parametara ruta, što omogućava dinamičko upravljanje podacima kroz URL-ove



Prikaz implmentacije 'Routinga' u glavni projekt

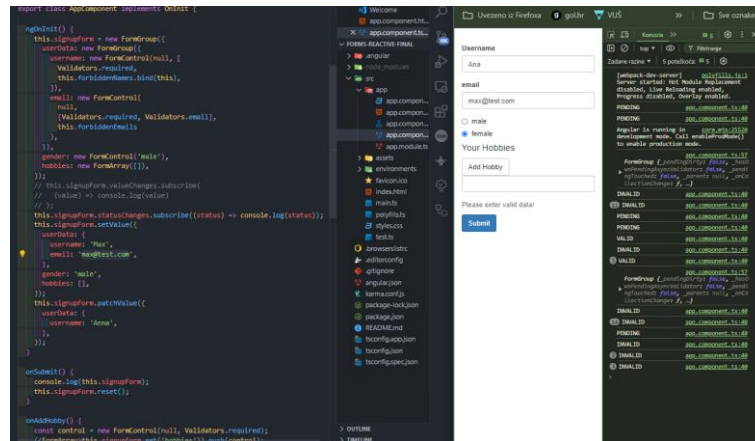
DAN 11.
17.10.2024

Prvi dio jedanaestog dana odvojili smo za rad s formama u Angular aplikacijama, fokusirani smo na temeljno razumijevanje kako Angular upravlja formama te kako se one implementiraju i koriste unutar aplikacija, razlika između predloška i reaktivnih pristupa te kada koristiti koji pristup.

Započeli sam s kreiranjem primjera forme, gdje sam prošli kroz proces kako postaviti formu u Angular aplikaciji.

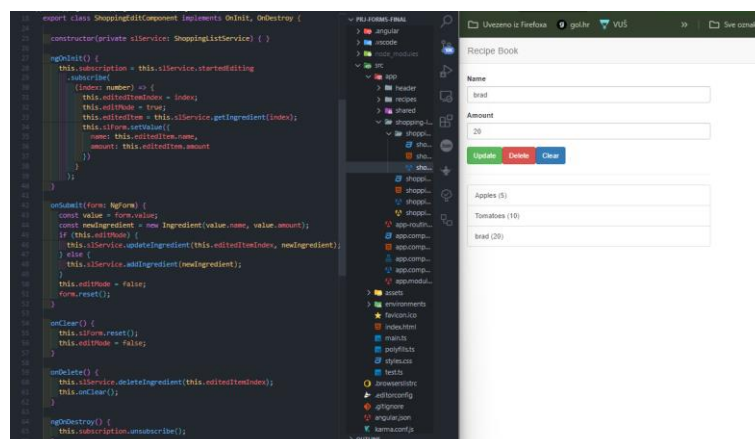
Posebno korisno bilo je raditi s FormArrays, gdje sam naučili kako omogućiti korisnicima da dinamički dodaju ili uklanjaju više ulaza unutar jedne forme.

Za kraj ovog predavanja koje smo poslušali, odrađujemo zadatak vezan za reagiranje na promjene vrijednosti unutar forme, što omogućava praćenje promjena u realnom vremenu i automatsko ažuriranje podataka u aplikaciji.



Projekt vezan za razumijevanje Angular formi

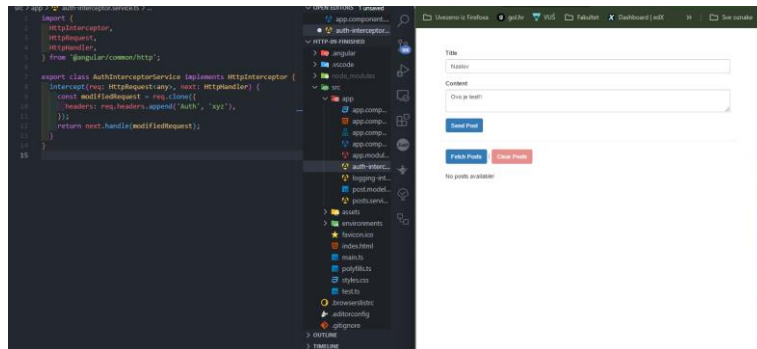
Drugi dio dana odvojili smo za kao i inače za implementiranje novo naučenih funkcionalnosti u naš projekt koji nas prati kroz trening, Započeli smo s dodavanjem forme za unos podataka na popis za kupnju, gdje sam postavio validaciju kako bih osigurao da korisnik unosi ispravne podatke. Dalje smo proširili funkcionalnost forme tako da korisnici mogu ažurirati postojeće stavke, resetirati formu i omogućiti korisnicima da izbrišu stavke iz popisa. Time smo priveli još jedan dan kraju.



Implementacija Angular formi u glavni projekt

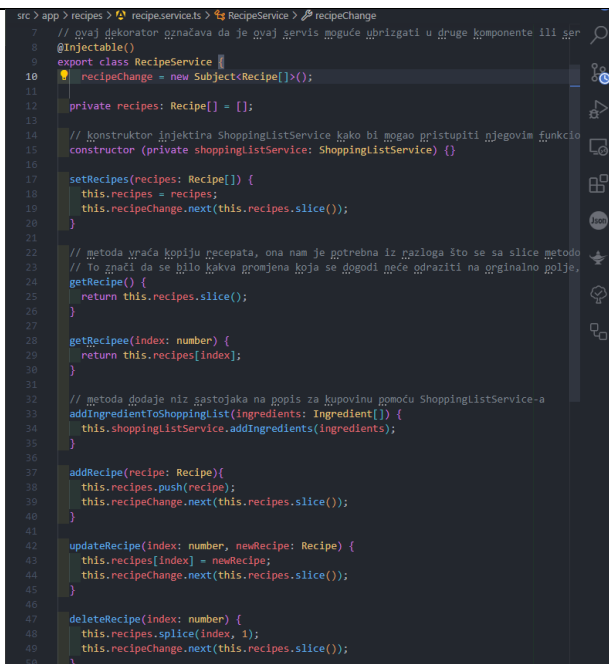
DAN 12.
18.10.2024

Danas krećemo sa poglavljem koje je namijenjeno Angular HTTP Client servisu za slanje zahtjeva prema API-ju. Kako dobiva odgovore, kako pravilno postaviti GET i POST zahtjeve, kako obraditi podatke koje API vraća. Koristili smo RxJS operator za transformaciju podataka nakon što smo ih dohvatili sa servera. Prošli smo kroz različite tehnike rukovanja pogreškama, uključujući korištenje Subjects za bolju kontrolu nad aplikacijom u slučaju greške. Za kraj poglavlja koristiti smo interceptore za manipulaciju zahtjevima i odgovorima prije nego što dođu do aplikacije, što je ključna tehnika za autentifikaciju i rukovanje podacima.



Projekt vezan za HTTP Client poglavlje

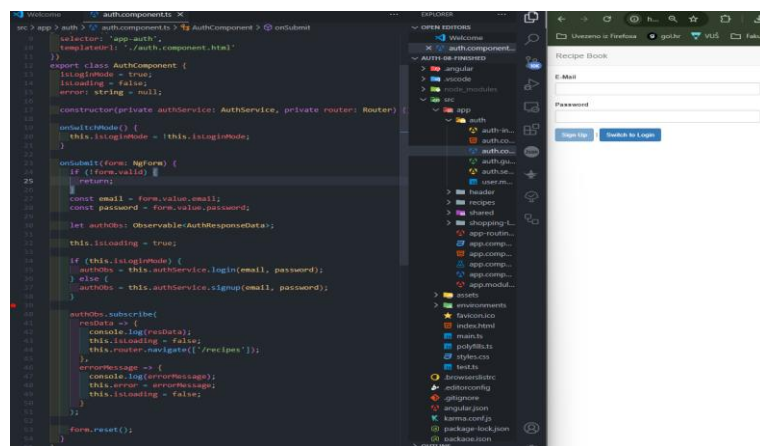
Kao i već standardi tempo koji vodimo zadnjih dana nakon lekcije krećemo sa implementacijom naučenog u naš projekt, za početak postavljamo backend-a koristeći Firebase. Nakon toga, postavili smo DataStorage servis, koji je odgovoran za pohranjivanje i dohvaćanje podataka iz Firebase baze podataka. Želimo pohraniti recepte u bazu podataka putem DataStorage servisa, što je ključno za upravljanje podacima u aplikaciji.

	 <pre> src > app > recipes > recipe.service.ts > RecipeService > recipeChange 7 // gvaaj dekorator gznacava da je gvaaj servis moguće ubrizgati u druge komponente ili ser 8 @Injectable() 9 export class RecipeService { 10 recipeChange = new Subject<Recipe[]>(); 11 12 private recipes: Recipe[] = []; 13 14 // konstruktor injektira ShoppingListService kako bi mogao pristupiti njegovim funkcio 15 constructor (private shoppingListService: ShoppingListService) {} 16 17 setRecipes(recipes: Recipe[]) { 18 this.recipes = recipes; 19 this.recipeChange.next(this.recipes.slice()); 20 } 21 22 // metoda vraća kopiju recepta, ona nam je potrebna iz razloga što se sa slice metodo 23 // to znači da se bilo kakva promjena koja se dogodi neće odraziti na originalno polje, 24 getRecipe() { 25 return this.recipes.slice(); 26 } 27 28 getRecipee(index: number) { 29 return this.recipes[index]; 30 } 31 32 // metoda dodaje niz sastojaka na popis za kupovinu pomoću ShoppingListService-a 33 addIngredientToShoppingList(ingredients: Ingredient[]) { 34 this.shoppingListService.addIngredients(ingredients); 35 } 36 37 addRecipe(recipe: Recipe) { 38 this.recipes.push(recipe); 39 this.recipeChange.next(this.recipes.slice()); 40 } 41 42 updateRecipe(index: number, newRecipe: Recipe) { 43 this.recipes[index] = newRecipe; 44 this.recipeChange.next(this.recipes.slice()); 45 } 46 47 deleteRecipe(index: number) { 48 this.recipes.splice(index, 1); 49 this.recipeChange.next(this.recipes.slice()); 50 } 51 } </pre> <i>Prikaz dodanog servisa koji komunicira sa bazom podataka</i>
--	--

DAN 13.
21.10.2024

Utorak je rezerviran za rad na implementaciji autentifikacije i zaštite ruta u Angular-u. Jedan od ključnih zadataka današnjeg dana bio je prebacivanje između različitih stanja autentifikacije, tj. između prijave i registracije. Sljedeći zadatak bi je prebaciti se na backend za prijenos podataka, što uključuje slanje podataka na server i primanje odgovora.

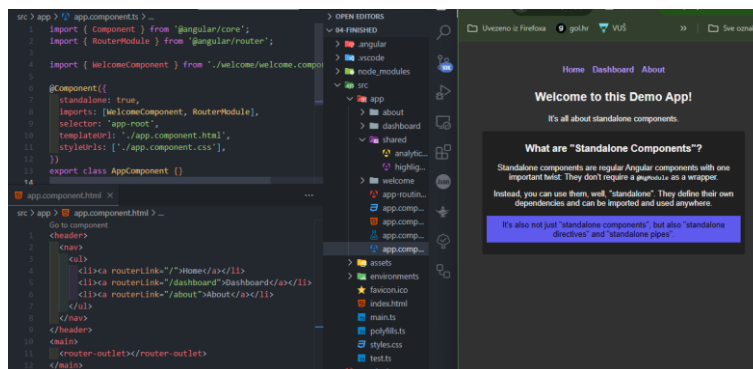
Implementiramo i funkcionalnosti poput odjave (Logout) i automatske prijave (Auto-Login), što omogućava korisnicima da ostanu prijavljeni čak i nakon zatvaranja aplikacije.



Implementacija autentifikacije u primjeru

Sljedeći zadatak bio je razumijevanje različitih pristupa za kreiranje i upravljanje komponentama unutar Angular aplikacije, naučili smo kako dodati novu Alert komponentu, koja se može dinamički prikazati ovisno o stanju aplikacije.

Zadnje dio dana iskoristili smo za rad sa samostalnim komponentama (Standalone Components) u Angularu, zašto su ove komponente korisne i kako se razlikuju od tradicionalnih Angular komponenti koje su povezane s modulima. Na kraju, istražujemo mogućnosti State Management unutar samostalnih komponenti

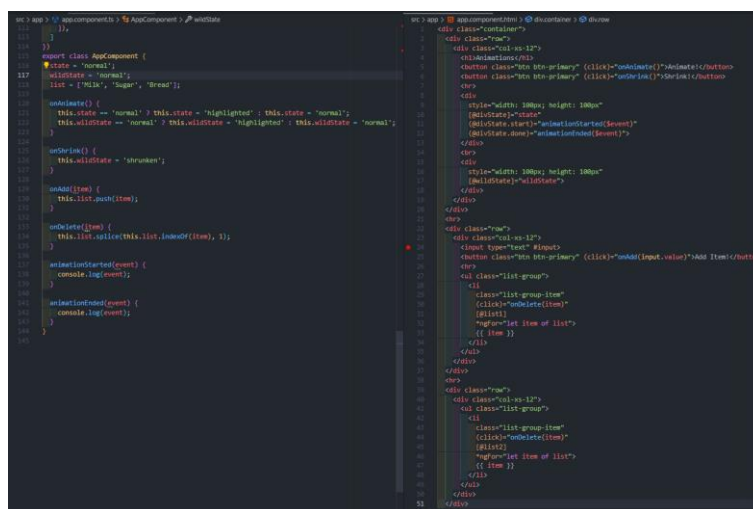


Prikaz Demo aplikacije

DAN 14.
22.10.2024

Za kraj Angular treninga ostalo je još nekoliko poglavlja koja ćemo proći. Prvo od njih je Angular Signals i upravljanje stanjem pomoću NgRx-a. Prolazimo kroz proces kako koristiti signale za praćenje i upravljanje vrijednostima unutar aplikacije.

Sljedeće poglavlje koje smo prošli bilo je praćenje kako animacije funkcioniraju u Angular-u od verzije 4+ te smo uz poglavlje postavili početni projekt za rad s animacijama. Otkrivamo i kako definirati okidače animacija i kako ih koristiti za prelazak između stanja u aplikaciji.

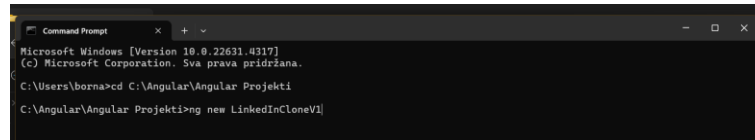


Primjer dodavanja animacija u projekt

	Nakon toga dolazimo do jediničnog testiranja u Angular aplikacijama. U ovoj lekciji prolazimo kroz osnovne principe jediničnog testiranja i zašto je ono važno za kvalitetu koda. Kroz lekcije vidimo kako kreirati i izvršavati testove pomoću alata koje pruža Angular CLI. Svladali smo kako testirati komponente, servise i njihovu međusobnu interakciju. Također istražili smo kako simulirati asinkrone zadatke unutar testova i kako koristiti alat za praćenje stanja testova. Ovdje smo završili sa Angular treningom i nastavljamo dalje u još jedan, zadnji trening a to je „RxJS In Practitice“ koji se bazira samo na teoretskom dijelu i primjerima bez ikakvih samostalnih projekata.
DAN 15. 23.10.2024	Petnaestim danom krećemo u novi, kratki trening i započinjemo s RxJS-om i reaktivnim dizajnom. Kroz prvi dio prolazimo kroz osnovne koncepte RxJS-a i zašto je reaktivno programiranje važno za razvoj modernih web aplikacija. Započinjemo s osnovama te kako one omogućavaju rukovanje asinkronim operacijama na učinkovit način upoznavanje U sljedećem dijelu fokus je na ključne RxJS operatore i njihovu primjenu u reaktivnom dizajnu. Koristimo operatore poput map za transformaciju podataka unutar tokova i kako se oni mogu implementirati unutar Angular aplikacija Rukovanje greškama u RxJS-u sadrži različite strategije za hvatanje i zamjenu grešaka, što je važno za osiguranje da aplikacija može pravilno obraditi neočekivane situacije. Jedna od ključnih lekcija bila je primjena retry operatora, koji omogućava ponovno pokretanje operacija u slučaju grešaka, čime se povećava otpornost aplikacije na povremene probleme poput prekida mreže. I za kraj kratkog treninga obratili smo pažnju na RxJS Subjecte i obrazac pohrane podataka (Store Pattern). Poseban naglasak bio je na primjeni store obrasca, gdje vidimo kako pohraniti i dohvaćati podatke unutar aplikacije koristeći RxJS i Angular. S ovim smo završili ovaj teoretski trening i sada smo spremni na stvaranje vlastitog projekta.
DAN 16. 24.10.2024	Za početak, projekt je osmišljen kao klon LinkedIn platforme, s fokusom na povezivanje profesionalaca, posebno freelancera, programera i drugih stručnjaka. Ključna ideja je omogućiti korisnicima da pronađu druge profesionalce u svojoj

blizini, te da putem profila unesu podatke o obrazovanju, vještinama, itd. Plan je da projekt obuhvaća standardne funkcionalnosti profesionalne društvene mreže kao što su pretraga korisnika, autentifikacija, upravljanje obavijestima, chat sustav, te prikaz korisničkih profila.

Fokus prvog dana rada na projektu bio je početno postavljanje, inicijalizacija i osnovna struktura. Proces smo započeli kreiranjem Angular aplikacije putem Angular CLI naredbe.

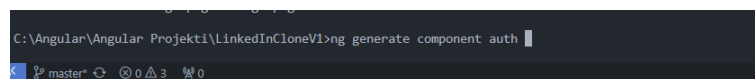


```
Microsoft Windows [Version 10.0.22631.4317]
(c) Microsoft Corporation. Sva prava pridržana.
C:\Users\borna>cd C:\Angular\Angular Projekti
C:\Angular\Angular Projekti>ng new LinkedInCloneV1
```

Angular CLI

Nakon kreiranja projekta, kreirali smo i ključne komponente aplikacije, koje će predstavljati njene osnovne dijelove, svaka komponenta implementira specifičan dio funkcionalnosti aplikacije:

- *AuthComponent* – autentifikacija korisnika (registracija, prijava, odjava)
- *HomeComponent* – prikaz korisničkog feeda, objava, i osnovnih informacija
- *ProfileComponent* – prikaz i uređivanje profila korisnika
- *SearchComponent* – omogućava pretragu korisnika prema određenim kriterijima te ih prikazuje na karti
- *ChatComponent* – tekstualni razgovor između korisnika s prikazom povijesti poruka
- *HeaderComponent* – glavni izbornik aplikacije s poljem za pretragu korisničkih profila



```
C:\Angular\Angular Projekti\LinkedInCloneV1>ng generate component auth
```

Stvaranje komponenti

Ovim načinom stvorili smo komponente koje odmah sadrže osnovne datoteke za logiku, prikaz i stilizaciju određene komponente. Sljedeći zadatak na koji smo se fokusirali bila je povezivanje s Firebase-om i konfiguracija. Odlučili smo koristiti *Firebase Console* za kreiranje novog Firebase projekta.

```
ng add @angular/fire
```

Naredba za dodavanje adaptera

Nakon što smo to postavili, krenuli smo sa postavljanjem Firebase konfiguracije u *environment.ts* datoteku, ona sadrži sve potrebne podatke za povezivanje aplikacije s Firebase projektom. Unutar *Firebase Console* stvorili smo novi projekt i prebacili smo podatke iz Firebase SDK u našu datoteku:

```
$ npm install firebase

Then, initialize Firebase and begin using the SDKs for the products you'd like to use.

// Import the functions you need from the SDKs you need
import { initializeApp } from "firebase/app";
import { getAnalytics } from "firebase/analytics";
// TODO: Add SDKs for Firebase products that you want to use
// https://firebase.google.com/docs/web/setup#available-libraries

// Your web app's Firebase configuration
// For Firebase JS SDK v7.20.0 and later, measurementId is optional
const firebaseConfig = {
  apiKey: "AIzaSyAupoqwxZ8MfLRs6cBPw99m7IKzAi3Z6Pg",
  authDomain: "linkedinklonv1.firebaseio.com",
  databaseURL: "https://linkedinklonv1-default-rtdb.firebaseio.com",
  projectId: "linkedinklonv1",
  storageBucket: "linkedinklonv1.appspot.com",
  messagingSenderId: "428100516891",
  appId: "1:428100516891:web:4cdf027183be720fd70cbe",
  measurementId: "G-RX8QNC1900"
};

// Initialize Firebase
const app = initializeApp(firebaseConfig);
const analytics = getAnalytics(app);
```

Firebase SDK

```
environments.ts
src > environments > environments.ts
1 import { getAnalytics } from "firebase/analytics";
2 import { initializeApp } from "firebase/app";
3
4 export const environment = {
5   production: false,
6   firebaseConfig: {
7     apiKey: "AIzaSyAupoqwxZ8MfLRs6cBPw99m7IKzAi3Z6Pg",
8     authDomain: "linkedinklonv1.firebaseio.com",
9     databaseURL: "https://linkedinklonv1-default-rtdb.firebaseio.com",
10    projectId: "linkedinklonv1",
11    storageBucket: "linkedinklonv1.appspot.com",
12    messagingSenderId: "428100516891",
13    appId: "1:428100516891:web:c7acfb7ccf10bc6d70cbe",
14    measurementId: "G-PI52GEPK0C"
15  },
16 };
17 const app = initializeApp(environment.firebaseConfig);
18 const analytics = getAnalytics(app);
19
```

Firebase konfiguracija

DAN 17.
25.10.2024

Sljedeći korak bio je **postavljanje autentifikacije korisnika**. *AuthComponenta* sadržavat će radnje poput registracije, prijave, odjave, provjere e-mail adrese te prijave preko Google računa. Za postavljanje autentifikacije kreirali smo

AuthService. To je ključna datoteka koja sadrži sve logove vezane za autentifikaciju korisnika.

```
C:\Angular\Angular Projekti\LinkedInCloneV1> ng generate service auth
```

Kreiranje servisa

Nakon što smo kreirali servis, u njega smo dodali funkcionalnosti koje su ključne za autentifikaciju, iz *angular/fire* biblioteke koristili smo *AngularFireAuth* koji omogućava Firebase autentifikaciju u Angular aplikacije, upravlja cijelim procesom i integrira se s Firebase backend-om kako bi se osiguralo sigurno upravljanje korisnicima.

Ključne funkcionalnosti servisa:

- *register()* – prima *email*, *password* i *userData*, te registrira korisnika putem *AngularFireAuth*.
- *login()* – provjerava podatke korisnika i prijavljuje ga, ako je sve uredu preusmjerava se na home stranicu
- *logout()* – postavlja korisnika na '*offline*', zatim poziva *signOut()* iz *AngularFireAuth*
- *googleSignIn()* – prijava putem Google-a pomoću *GoogleAuthProvider*, ako je prijava uspješna preusmjerava se na home stranicu, podaci korisnika se pohranjuju i ažuriraju u Firestore-u

Forma za registraciju i prijavu koristi Angular-ov reaktivni pristup (*ReactiveForms*) sa *formGroup* nazivom *authForm*. Obrazac se šalje prilikom *ngSubmit* događaja:

```
</>  
<h6 class="text-center">Stay updated on your professional world</h6>  
<form [formGroup]="authForm" (ngSubmit)="onSubmit()">  
  <div *ngIf="!isLoginMode">
```

Prikaz funkcionalnosti forme

Kao jednu od najvažnijih stvari, implementirali smo metodu za rad sa *Reactive Formom* (*initializeForm()*) kao i rukovanje sa prijavom i registracijom

```
onSubmit(): void {
  if (this.authForm.invalid && !this.isLoginMode) {
    this.authForm.markAllAsTouched();
    return;
  }

  const { email, password } = this.authForm.value;

  if (this.isLoginMode) {
    this.handleLogin(email, password);
  } else {
    this.authService
      .checkEmailExists(email)
      .then((emailExists) => {
        if (emailExists) {
          this.emailExists = true;
          this.authForm.get('email').setErrors({ emailExists: true });
        } else {
          this.emailExists = false;
          this.registerUser();
        }
      })
      .catch((error) => {
        console.error('Error during email check: ', error);
      });
  }
}
```

```
private initializeForm(): void {
  this.authForm = this.fb.group({
    firstName: ['', Validators.required],
    lastName: ['', Validators.required],
    email: ['', [Validators.required, Validators.email]],
    password: ['', [Validators.required, Validators.minLength(6)]],
    confirmPassword: [
      '',
      [Validators.required, this.matchValues('password')],
    ],
    contact: ['', [Validators.required, Validators.pattern(/^\d{4}+$/)]],
    address: ['', Validators.required],
    education: ['', Validators.required],
    skills: ['', Validators.required],
    bio: [''],
    profileImage: [''],
  });
}
```

Auth.component.ts

Login

Stay updated on your professional world

Email

Password

[Forgot password?](#)

Login

Join Now

 Continue with Google

Forma prijave u aplikaciju

	Stay updated on your professional world Odaberi datoteku Nije odabrana niti jedna datoteka. Profile Image First Name Last Name Contact Address Bio Email Password Confirm Password Register Prikaz forme za registraciju novog korisnika
DAN 18. 28.10.2024	S trećim danom rada na LinkedIn Clon aplikaciji fokusirali smo se na postavljanje osnovne navigacije i zaštite ruta. Prvi zadatak bio je dodavanje usmjerivača i glavne rute u novu datoteku koju smo stvorili nazivom <code>app.routes.ts</code>, kako bi se korisnici mogli kretati kroz različite stranice unutar aplikacije. Postavljanje ruta u aplikaciji definiramo koristeći <code>Routes</code> sučelje iz <code>@angular/router</code>. Da bi Angular prepoznao ove rute, <code>routes</code> sučelje mora biti povezano s <code>RouterModule</code> u <code>app.module.ts</code> datoteci. Korištenjem <code>RouterModule.forRoot(routes)</code>, aplikacija je konfigurirana s definiranim rutama, omogućujući navigaciju i pristup komponentama putem URL-a <pre> imports: [BrowserModule, RouterModule.forRoot(routes), </pre> <i>Dodavanje RouterModule u AppModule.ts datoteku</i> Sljedeća implementacija koja je jako povezana sa osnovnom navigacijom je i zaštita tih ruta po kojim se krećemo. Kako bi osigurali sigurnost podataka, implementirali smo <code>AuthGuard</code> koji će sprječavati neautorizirane korisnike u pristupu zaštićenim rutama. <code>AuthGuard</code> implementira Angular-ovo <code>CanActivate</code> sučelje, koje definira može li korisnik pristupiti određenoj ruti.

```

src > app > auth.guard.ts > AuthGuard > canActivate
1 import { Injectable } from '@angular/core';
2 import {
3   ActivatedRouteSnapshot,
4   CanActivate,
5   Router,
6   RouterStateSnapshot,
7   UrlTree,
8 } from '@angular/router';
9 import { Observable, map, take } from 'rxjs';
10 import { AuthService } from '../services/auth.service';
11
12 @Injectable({
13   providedIn: 'root',
14 })
15 export class AuthGuard implements CanActivate {
16   constructor(private authService: AuthService, private router: Router) {}
17
18   canActivate(
19     route: ActivatedRouteSnapshot,
20     state: RouterStateSnapshot
21   ): Observable<boolean | UrlTree> {
22     return this.authService.getUser().pipe(
23       take(1),
24       map((user) => {
25         if (user) {
26           return true;
27         } else {
28           return this.router.createUrlTree(['login']);
29         }
30       })
31     );
32   }
33 }
34

```

AuthGuard implementacija

Nakon što smo implementirali *AuthGuard*, dodali smo ga u *app.routes.ts* datoteku kako bi zaštitili rute koje smo kreirali:

```

src > app > app.routes.ts > routes
1 import { Routes } from '@angular/router';
2 import { AuthGuard } from '../auth.guard';
3 import { AuthComponent } from '../auth/auth.component';
4 import { ChatComponent } from '../chat/chat.component';
5 import { HomeComponent } from '../home/home.component';
6 import { ProfileComponent } from '../profile/profile.component';
7 import { SearchComponent } from '../search/search.component';
8
9 export const routes: Routes = [
10   { path: '', redirectTo: 'login', pathMatch: 'full' },
11   { path: 'home', component: HomeComponent, canActivate: [AuthGuard] },
12   { path: 'profile', component: ProfileComponent, canActivate: [AuthGuard] },
13   { path: 'search', component: SearchComponent, canActivate: [AuthGuard] },
14   { path: 'login', component: AuthComponent },
15   { path: 'register', component: AuthComponent },
16   { path: 'chat', component: ChatComponent, canActivate: [AuthGuard] },
17 ];
18

```

App.routes.ts datoteka

DAN 19.
29.10.2024

Danas je na red došla **HeaderComponent**, ova komponenta služi nam kao zaglavlje za Angular aplikaciju, pružajući navigaciju između stranica.

Glavne funkcionalnosti koje smo uveli u *HeaderComponent*-u su:

1. Navigacijski linkovi i ikone:
 - Home – gumb koji vodi na *HomeComponent*
 - Profile – gumb koji vodi na *ProfileComponent*
 - Search – gumb koji vodi na *SearchComponent*
 - Chat – gumb koji vodi na *ChatComponent*
2. Polje za pretragu korisnika

3. Gumb za odjavu

Koristeći RouterLink, svaki link vodi korisnika na određenu rutu bez osvježavanja stranice:

```
<a
  class="nav-link"
  routerLink="/home"
  routerLinkActive="active"
  [routerLinkActiveOptions]="{ exact: true }"
>
  <ion-icon name="home-outline"></ion-icon>
  Home
</a>
```

Navigacijski 'Home' link

Sljedeća stavka koju smo dodali u *HeaderComponent* bila je *searchBar* tj. polje za pretragu korisnika, koja prikazuje rezultate u padajućem meniju, te pretražuje korisnike prema *searchQuery*.

```
<input
  class="form-control me-2"
  type="search"
  placeholder="Search"
  aria-label="Search"
  [(ngModel)]="searchQuery"
  name="searchQuery"
  (input)="searchUsers()"
/>
<div
  class="dropdown-menu show search-dropdown"
  *ngIf="searchResults.length > 0"
>
```

Polje za pretragu korisnika

I na kraju dodali smo gumb koji sadrži funkcionalnosti odjave korisnika, koji poziva metodu za odjavu iz *AuthService*-a

Nakon toga kreirali smo novi **servis UserService** – ovaj servis koristit će nam za dohvaćanje podataka o korisnicima iz baze podataka. U *UserService* dodali smo metodu koja će nam koristiti kod pretraživanja korisnika u *HeaderComponent*-i.

```

searchUsers(query: string): Observable<any[]> {
  return this.firestore
    .collection('users')
    .valueChanges()
    .pipe(
      map((users) =>
        users.filter(
          (user: any) =>
            (user.firstName &&
              user.firstName.toLowerCase().includes(query.toLowerCase())) ||
            (user.lastName &&
              user.lastName.toLowerCase().includes(query.toLowerCase()))
        )
      )
    );
}

```

UserService metoda searchUsers

Na kraju **stvorili smo i HeaderService** i u njemu smo primijenili metode za upravljanje vidljivosti zaglavlja u aplikaciji, tj. prilikom prijave ili registracije navigacijska traka neće biti vidljiva.

```

import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root',
})
export class HeaderService {
  private _isHeaderVisible = true;

  hide() {
    this._isHeaderVisible = false;
  }

  show() {
    this._isHeaderVisible = true;
  }

  get isHeaderVisible(): boolean {
    return this._isHeaderVisible;
  }

  constructor() {}
}

```

Struktura HeaderService



Izgled HeaderComponent

DAN 20.
30.10.2024

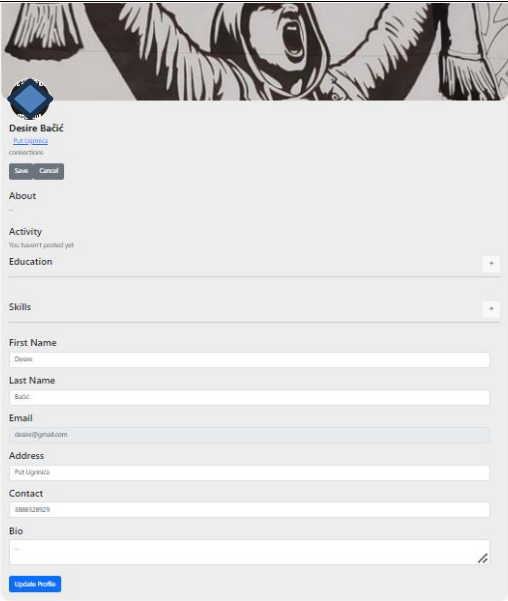
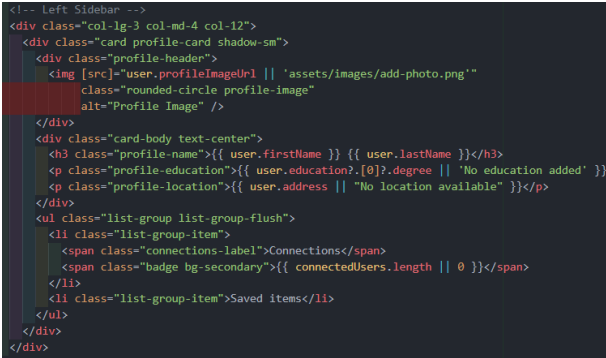
Petim danom kreiranja Angular aplikacije, zadatak je bio implementirati i postaviti profil korisnika i prikaz njegovih podataka. **ProfileComponent** prikazuje i ažurira osnovne podatke o korisniku

Prikaz ProfileComponent

Uz kreiranje *ProfilComponent*, **kreirali smo i ProfileService** koji upravlja podacima o profilu korisnika u Firebase bazi i pruža funkcije za pretraživanje korisnika i ažuriranje podataka o korisniku.

Prikaz getUserData()

Prikaz `updateUserData()`

	 <i>Prikaz ProfileComponent</i>
DAN 21. 31.10.2024	Zadatak šestog dana rada na Angular aplikaciji rezerviran je za postavljanje HomeComponent-e, te njenih funkcionalnosti. Lijevi dio komponente: - Prikazuje sliku profila, ime, obrazovanje i lokaciju korisnika. Podaci se prikazuju kroz user objekt koji je učitao u bazi  <i>Lijevi dio HomeComponent-e</i> Srednji dio komponente prikazuje dio koji se odnosi na kreiranje postova i prikaz objava: Kreiranje objava:  <i>Srednji dio komponente, određen za kreiranje objave</i>

Prikaz objava:

```
<!-- Display Posts -->
<div *ngFor="let post of posts | async" class="card mb-3 post-card">
  <div class="card-body">
    <div class="d-flex">
      <img [src]="post.userProfileImageUrl" class="rounded-circle post-profile-image me-3" alt="Profile Image"
        (click)="goToProfile(post.userId)" />
    </div>
    <div>
      <h6 class="card-title mb-0" (click)="goToProfile(post.userId)">{{ post.userName }}</h6>
      <p class="post-date">{{ post.timestamp | date : 'short' }}</p>
    </div>
    <hr />
    <p class="card-text">{{ post.content }}</p>
    <div class="d-flex justify-content-between align-items-center">
      <button (click)="toggleLike(post)" class="like-button" [class.liked]="post.likedBy.includes(user.uid)">
        <i [ngClass]="post.likedBy.includes(user.uid) ? 'bi bi-hand-thumbs-up-fill' : 'bi bi-hand-thumbs-up'" />
        <span>{{ post.likes }}</span>
      </button>
    </div>
  </div>
</div>
</div>
```

Srednji dio komponente, određen za prikaz objava

Ideja je da desni dio ostavimo praznim kako bi kasnije tamo implementirali prikaz liste povezanih korisnika.

TypeScript datoteka, neke od glavnih metoda:

- *ngOnInit()* - prati promjene u autentifikaciji korisnika i učitava podatke o korisniku iz Firestore, objave se učitavaju i sortiraju po vremenu i prikazuju se u *feedu*.

```
ngOnInit(): void {
  this.loadData();

  this.afAuth.authState.subscribe((user) => {
    if (user) {
      this.firestore
        .collection('users')
        .doc(user.uid)
        .valueChanges()
        .subscribe(userData: any => {
          if (userData) {
            this.user = {
              ...userData,
              uid: user.uid,
            };
            this.userLoaded = true;
            if (this.user.connections && this.user.connections.length > 0) {
              this.loadConnectedUsers(this.user.connections);
            }
          }
        });
      if (this.user.connections && this.user.connections.length > 0) {
        this.loadConnectedUsers(this.user.connections);
      } else {
        this.user.connections = [];
      }
    }
  });

  // Dohvaćanje objava iz Firestore baze podataka
  this.posts = this.firestore
    .collection('posts', (ref) => ref.orderBy('timestamp', 'desc'))
    .snapshotChanges()
    .pipe(
      map((actions) =>
        actions.map((a: any) => {
          const data = a.payload.doc.data() as Post;
          const id = a.payload.doc.id;
          return {
            id,
            ...data,
            likedBy: data.likedBy || [],
          };
        })
      ),
      // Pretvorba Firestore Timestamp-a u JavaScript Date
      map((posts: any) =>
        posts.map((post: any) => ({
          ...post,
          timestamp: post.timestamptoDate(),
        }))
      )
    );
}
```

Prikaz ngOnInit()

- *submitPost()* – kreiran novu objavu i dodaje ju u Firebase bazu, prazni polje za unos nakon uspješnog dodavanja objave


```

submitPost() {
  if (this.postContent.trim() && this.user && this.user.uid) {
    const post = {
      content: this.postContent,
      timestamp: new Date(),
      userId: this.user.uid,
      userName: `${this.user.firstName} ${this.user.lastName}`,
      userProfileImageUrl:
        this.user.profileImageUrl || 'assets/images/add-photo.png',
      likes: 0,
      likedBy: [],
    };

    this.firestore
      .collection('posts')
      .add(post)
      .then(() => {
        console.log('Post added successfully');
        this.postContent = '';
      })
      .catch((error) => {
        console.error('Error adding post: ', error);
      });
  }
}

```

Prikaz onSubmit()

- *toggleLike()* – provjerava da li je korisnik već označio objavu sa 'sviđa mi se', ako je uklanja oznaku, ako ga nije već označio dodaje oznaku.

```

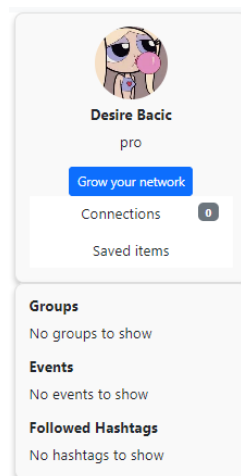
toggleLike(post: Post) {
  const postRef = this.firestore.collection('posts').doc(post.id);

  // Provjeravamo trenutno stanje lajkova
  postRef.get().subscribe((docSnapshot) => {
    if (docSnapshot.exists) {
      const postData = docSnapshot.data() as Post;
      const currentLikes = postData?.likes || 0;
      const likedBy = postData?.likedBy || []; // Popis korisničkih ID-ova koji su

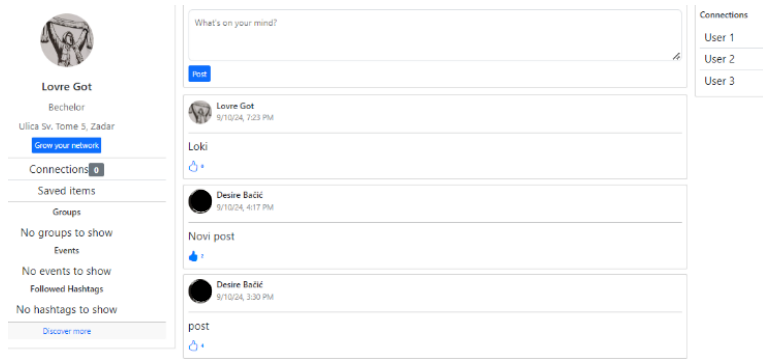
      // Povećavamo broj lajkova
      if (likedBy.includes(this.user.uid)) {
        // Ako je korisnik već lajkao post, makni njegov lajk
        const updatedLikedBy = likedBy.filter(
          (userId: string) => userId !== this.user.uid
        );
        postRef
          .update({
            likes: currentLikes - 1,
            likedBy: updatedLikedBy,
          })
          .catch((error) => {
            console.error('Error removing like: ', error);
          });
      } else {
        // Ako korisnik nije lajkao, dodaj njegov lajk
        postRef
          .update({
            likes: currentLikes + 1,
            likedBy: [...likedBy, this.user.uid],
          })
          .catch((error) => {
            console.error('Error liking post: ', error);
          });
      }
    }
  });
}

```

Prikaz toggleLike() metode



Lijevi sidebar HomeComponent

	 Prikaz trenutnog stanja HomeComponent
--	---

DAN 22.
01.11.2024

Danas smo **implementirali SearchComponent**, kao i novi servis kojeg smo nazvali **Geocoding**. Komponenta omogućava korisnicima pretraživanje drugih korisnika na temelju lokacije, radijusa i vještina. Komponenta također koristi Google Map API za prikaz korisnika na karti i koristi servis za dobivanje geografskih koordinata na temelju unesenih adresa.

Glavne funkcionalnosti ove komponente su sljedeće:

- Kada korisnik unese sve potrebno za pretragu, `searchUsers()` metoda poziva `geocodingService.getCoordinates()`, koji vraća koordinate lokacije.

```
searchUsers() {
  this.searchAttempted = true; // Označava da je pretraga pokrenuta

  const skillInput: string = this.skillControl.value ?? '';

  // Provjera je li lokacija unesena
  if (!this.location) {
    this.locationError = true; // Prikaz pogreške za praznu lokaciju
    return;
  }
  this.locationError = false;

  this.geocodingService.getCoordinates(this.location).subscribe(
    (coords) => {
      console.log('Coordinates:', coords);
      this.center = coords;
      this.filteredUsers = [];

      this.users.forEach((user) => {
        if (!user.position || !user.position.lat || !user.position.lng) {
          console.warn(
            `User ${
              user.firstName || 'Unnamed User'
            } does not have valid position data.`
          );
          return;
        }

        const distance = this.calculateDistance(this.center, user.position);

        const skillsArray = Array.isArray(user.skills) ? user.skills : [];

        const hasMatchingSkill = skillsArray.some((skill: any) => {
          const skillName =
            typeof skill === 'string' ? skill : skill.skillName;
          return (
            skillName &&
            skillName.toLowerCase().includes(skillInput.toLowerCase())
          );
        });
      });
    }
  );
}
```

Prikaz searchUsers() metode

- `calculateDistance()` – računa udaljenost između unesene lokacije i pozicije korisnika, koristeći *Haversineovu* formulu

```

calculateDistance(
  center: google.maps.LatLngLiteral,
  position: google.maps.LatLngLiteral
): number {
  const R = 6371;
  const dLat = this.deg2rad(position.lat - center.lat);
  const dLng = this.deg2rad(position.lng - center.lng);
  const a =
    Math.sin(dLat / 2) * Math.sin(dLat / 2) +
    Math.cos(this.deg2rad(center.lat)) *
    Math.cos(this.deg2rad(position.lat)) *
    Math.sin(dLng / 2) *
    Math.sin(dLng / 2);
  const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
  const distance = R * c;
  return distance;
}

```

Prikaz calculateDistance() metode

- prikaz rezultata na karti: karta prikazuje korisnike na osnovu njihovih *lat* i *lng* koordinata.

Geocoding.service.ts datoteka omogućava prevođenje adresa u geografske koordinate i obrnuto.

```

import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { Observable, catchError, map, of } from 'rxjs';

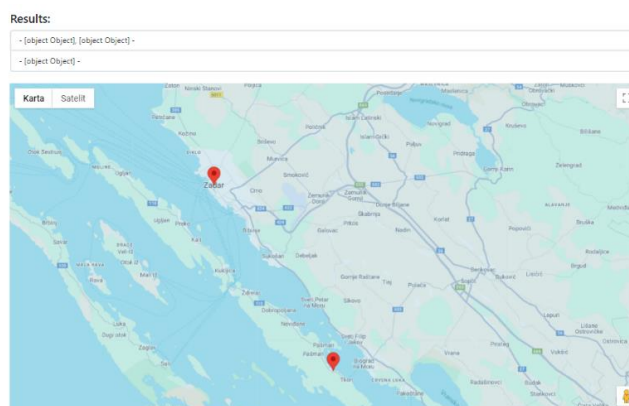
@Injectable({
  providedIn: 'root'
})
export class GeocodingService {
  private apiKey = 'AIzaSyCp37IP20yvJ0y0MjKlabl5-C-RaTvsLD4';

  constructor(private http: HttpClient) {}

  getCoordinates(address: string): Observable<any> {
    const url = `https://maps.googleapis.com/maps/api/geocode/json?address=${encodeURIComponent(address)}&key=${this.apiKey}`;
    return this.http.get(url).pipe(
      map((response: any) => {
        console.log('Geocoding API response:', response);
        if (response.status === 'OK' && response.results.length > 0) {
          return response.results[0].geometry.location;
        }
        throw new Error('Location not found');
      }),
      catchError((error) => {
        console.error('Error fetching coordinates:', error);
        return of({ lat: 0, lng: 0 });
      })
    );
  }
}

```

Prikaz getCoordinates()



Implementacija Google Maps

	Location Zadar Radius (km) 500 Skill Flutter Search Results: Lovre Got Skills: Angular, JS, Flutter Location: Ul. Ivana Mažuranića 7, 23000, Zadar, Hrvatska Karta Satelit  <i>SearchComponent forma</i>
DAN 23. 04.11.2024	Nakon kreiranja ChatComponent, vrijeme je da ju i implementiramo. Danas smo u komponentu koja će biti zadužena za razmjenu poruka i upravljanje razgovorima postavili osnovnu strukturu. U listi razgovora, koja se nalazi u lijevoj koloni prikazuju se kartice sa informacijama korisnika s kojim razmjenjujemo poruke, ta kartica prikazuje datum posljednje poruke i sadržaj posljednje poruke kao i broj nepročitanih poruka u oznaci (<i>badge</i>). <pre> <div class="col-md-4 conversation-list"> <h4>Conversations</h4> <ul class="list-group"> <li *ngFor="let conversation of conversations\$ async" class="list-group-item conversation-item d-flex justify-content-between" (click)="selectConversation(conversation)" [ngClass]="{ unread: conversation.unread }" > <div class="d-flex align-items-center"> <div class="conversation-details"> <div class="conversation-name"> {{ getParticipantName(conversation) }} <!-- Use the conversation-specific online status --> </div> <div class="conversation-snippet"> {{ conversation.lastMessage.content }} </div> </div> </div> <div class="conversation-meta d-flex align-items-center"> <div class="conversation-date me-2"> {{ conversation.lastMessage.timestamp date: 'MM d' }} </div> <div *ngIf="conversation.unreadCount" class="badge bg-primary rounded-pill me-2"> {{ conversation.unreadCount }} </div> </div> </div> </pre> <i>Prikaz strukture liste razgovora</i> Sekcija za poruke prikazuje se na način da odaberete razgovor i u desnom dijelu nalazi se povijest razgovora tj. sve poruke između ta dva korisnika te polje za unos

nove tekstualne poruke sa gumbom za slanje. Poruke se prikazuju kao *sent* (poslane) ili *received* (primljene).

```
<!-- Message List -->
<div class="message-list" #messageList>
  <!-- Loop through messages -->
  <div
    *ngFor="let message of messages$ | async"
    class="message-item"
    [ngClass]="{
      sent: message.senderId === currentUser?.uid,
      received: message.senderId !== currentUser?.uid
    }"
  >
    <!-- Message content -->
  </div>
</div>
```

Stanje tekstualne poruke

Implementiran je i **ChatService** koji upravlja svim funkcionalnostima vezanim za komunikaciju, kao što su, jedne od glavnih, dohvaćanje razgovora i upravljanje porukama.

- Dohvaćanje razgovora (getConversations())

```
getConversations(userId: string): Observable<Conversation[]> {
  return this.firestore
    .collection<Conversation>('conversations', (ref) =>
      ref.where('participants', 'array-contains', userId)
    )
    .snapshotChanges()
    .pipe(
      switchMap((actions) => {
        const conversations = actions.map((a) => {
          const data = a.payload.doc.data() as Conversation;
          const id = a.payload.doc.id;
          // Convert Timestamp to Date
          if (
            data.lastMessage &&
            data.lastMessage.timestamp &&
            data.lastMessage.timestamp.toDate()
          ) {
            data.lastMessage.timestamp = data.lastMessage.timestamp.toDate();
          }
          return { ...data, id };
        });
        const unreadCounts = conversations.map((conversation) =>
          this.firestore
            .collection(`conversations/${conversation.id}/messages`, (ref) =>
              ref.where('senderId', '!=', userId).where('read', '==', false)
            )
            .valueChanges()
            .pipe(
              map((messages) => ({
                conversationId: conversation.id,
                count: messages.length,
              }))
            )
        );
      })
    );
}
```

Prikaz metode odgovorne za dohvaćanje razgovora

- Dohvaćanje i slanje poruka (getMessages() i sendMessage())

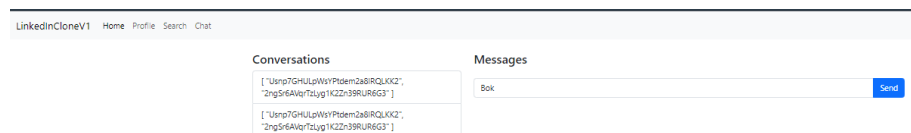
```

// Get messages for a specific conversation
getMessages(conversationId: string): Observable<Message[]> {
  return this.firestore
    .collection<Message>(`conversations/${conversationId}/messages`, (ref) =>
      ref.orderBy('timestamp')
    )
    .valueChanges()
    .pipe(
      map((messages) =>
        messages.map((message) => ({
          ...message,
          timestamp: (message.timestamp as any).toDate(), // Convert Firebase
        }))
      )
    );
}

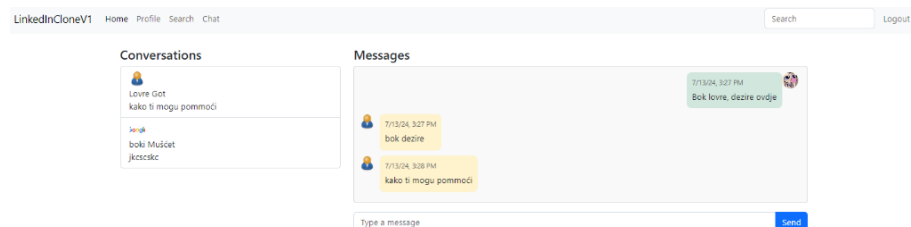
// Send a new message to a conversation
sendMessage(conversationId: string, content: string, senderId: string): void {
  const message: Message = {
    content,
    timestamp: new Date(),
    senderId,
    read: false,
  };
  this.firestore
    .collection(`conversations/${conversationId}/messages`)
    .add(message);
  this.firestore.collection('conversations').doc(conversationId).update({
    lastMessage: message,
  });
}

```

Prikaz metoda za slanje i dohvaćanje poruka



Početna implementacija ChatComponente



Razvoj ChatComponente

DAN 24.
05.11.2024

Današnji zadatak bio je implementirati sustav za postavljanje profilne slike korisnika i pozadinske slike preko odvojene komponente koja će surađivati sa *ProfileComponent* i njoj će koristiti. S toga smo stvorili **Image-modalComonent**. Ova komponenta koristi nam kao modalni dijalog koji omogućava korisnicima pregled, prijenos i brisanje profilne ili pozadinske slike.

Glavne funkcionalnosti ove komponente:

- Modal prikazuje trenutno profilnu ili pozadinsku sliku korisnika.
- Odabir nove slike (*onFileSelected()*) – kada korisnik odabere novu datoteku, metoda učitava sliku i prikazuje je kao pregled.

```

ngOnInit(): void {}

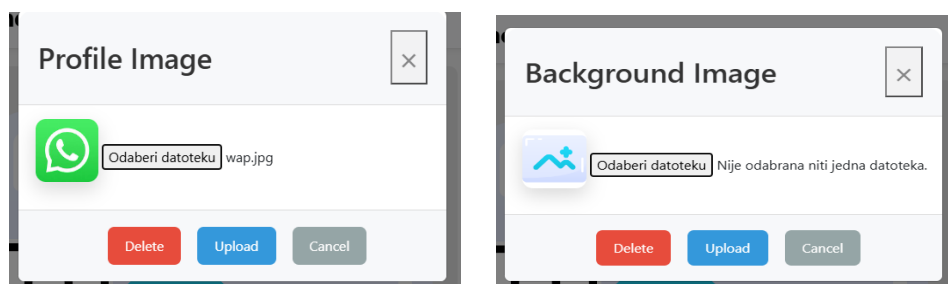
onFileSelected(event: any): void {
  const file: File = event.target.files[0];
  if (file) {
    this.selectedFile = file;

    const reader = new FileReader();
    reader.onload = (e: any) => {
      this.previewUrl = e.target.result;
    };
    reader.readAsDataURL(file);
  }
}

```

Prikaz metode *onFileSelected*

- Prijenos slike (*uploadImage()*) – omogućava prijenos slike u Firestore Storage i ažuriranje Firestore dokumenta korisnika s novim URL-om slike
- Brisanje slike (*deleteImage()*) – briše trenutnu sliku korisnika i vraća zadanu sliku



Prikaz *ImageModalComponent*

Ažuriranje slike nakon promjene ili brisanja

- Pretplatom na *onImageChange()* i *onDelete()* događaje, *ProfilComponent* ažurira URL slike u stvarnom vremenu bez potrebe za ponovnim učitavanjem

DAN 25.
06.11.2024

Današnji zadatak je dodavanje još jedne komponente koja će biti modalni dijalog. **EducationModalComponent** omogućava korisnicima dodavanje i uređivanje podataka o obrazovanju kroz modalni dijalog.

Komponenta koristi *Angular Reactive Forms* za upravljanje obrazovnim podacima i *ng-bootstrap* za prikazivanje modalnog dijaloga.

Ng-bootstrap je biblioteka za dodavanje *Bootstrap* stilova i funkcionalnosti u Angular aplikacije, osiguravajući dizajn i olakšavajući rad s korisničkim sučeljem bez dodatnog sloja *JavaScript* koda.

Glavne funkcionalnosti komponente:

```
// Metoda za inicijalizaciju formulara
private initializeForm(): void {
  this.educationForm = this.fb.group({
    schoolName: ['', Validators.required],
    degree: ['', Validators.required],
    fieldOfStudy: ['', Validators.required],
    startDate: [''],
    endDate: [''],
    grade: [''],
    activities: [''],
    description: [''],
  });
}
```

Prikaz strukture obrasca

```
<h5 class="modal-title">
  {{ education ? "Edit Education" : "Add Education" }}
</h5>
<button type="button" class="close" aria-label="Close" (click)="close()">
  <span aria-hidden="true">&times;</span>
</button>
</div>
```

Prikaz forme ovisno o odabiru korisnika

Ova forma namijenjena je za korištenje u *ProfilComponenti*, korisnik otvora modal putem metode *openEducationModal*. U *ProfileComponenti* korisnik ima mogućnost dodavanja, uklanjanja i ažuriranja forme koja se odnosi na obrazovanje.

Prikaz dijaloga za dodavanje obrazovanja

DAN 26.
07.11.2024

Dvadesetšestim danom praktične nastave ponovno smo stvorili novu komponentu imenom **SkillsModalComponent** i servis odgovoran za upravljanje vještinama **SkillsService**.

Ova komponenta omogućava korisnicima dodavanje ili ažuriranje putem modalnog prozora. Koristi se u *ProfileComponent* za prilagodbu korisničkog seta vještina.

Struktura forme pruža korisničko sučelje za unos i za ažuriranje vještina, ovisno o tome je li modal otvoren za dodavanje nove vještine ili za ažuriranje postojeće

Glavne funkcionalnosti:

- *Angular FormGroup* obrazac sa poljima za podatke

```
constructor(public activeModal: NgbActiveModal, private fb: FormBuilder) {  
  this.skillForm = this.fb.group({  
    skillName: ['', Validators.required],  
    relatedField: [''],  
    experience: ['', Validators.required],  
  });  
}
```

Prikaz strukture FormGroup obrasca

- Provjera zadanosti *skills* varijable

```
ngOnInit(): void {  
  if (this.skill) {  
    this.isUpdateMode = true;  
    this.skillForm.patchValue(this.skill);  
  }  
  
  // Pratimo promene u formi  
  this.skillForm.valueChanges.subscribe(() => {  
    this.checkForChanges();  
  });  
}
```

Provjera skills varijable

- Korištenje u *ProfileComponenti*

The image shows a user interface for managing skills. On the left, there is a modal dialog titled 'Add Skill' with a close button (X). Inside the dialog, there are three input fields: 'Skill Name', 'Related Field', and 'Experience'. At the bottom of the dialog are 'Save' and 'Cancel' buttons. On the right, there is a section titled 'Skills' which contains a single skill entry: 'Angular'.

Prikaz modal dijaloga za dodavanje nove vještine

Ostatak radnog dana iskoristili smo za implementaciju *SkillsService* koji je odgovoran za dohvaćanje popisa svih vještina iz *users* kolekcije u *Firestore* i pohranjuje ih kao *BehaviorSubject*.

- *FetchSkills()* metoda koristi *valueChanges()* za dohvaćanje korisničkih vještina, koje se pohranjuju u *Set* kako bi se uklonili duplicirani podatci. Vještine su sortirane i pretvorene u niz.

```
export class SkillsService {
  private skillsSubject: BehaviorSubject<string[]> = new BehaviorSubject<string[]>();
  public skills$: Observable<string[]> = this.skillsSubject.asObservable();

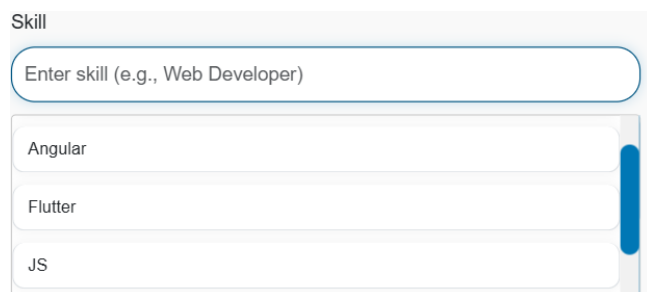
  constructor(private firestore: AngularFirestore) {
    this.fetchSkills();
  }

  private fetchSkills() {
    this.firestore.collection('users').valueChanges().pipe(
      map(users => {
        const skillsSet = new Set<string>();
        users.forEach((user: any) => {
          if (user.skills && Array.isArray(user.skills)) {
            user.skills.forEach((skill: any) => {
              if (typeof skill === 'string') {
                skillsSet.add(skill.trim());
              } else if (typeof skill === 'object' && skill.skillName) {
                skillsSet.add(skill.skillName.trim());
              }
            });
          }
        });
        return Array.from(skillsSet).sort();
      })
    ).subscribe(skills => {
      this.skillsSubject.next(skills);
    });
  }

  public getSkills(): string[] {
    return this.skillsSubject.value;
  }
}
```

Prikaz *SkillsService-a*

Korištenje *skillsService-a* u *SearchComponent* – servis omogućava dohvaćanje jedinstvenog popisa svih vještina koje su korisnici dodali. *SearchComponent-a* koristi ovaj servis kako bi korisniku ponudio popis dostupnih vještina prilikom filtriranja korisnika u formi za pretragu.



Popis unesenih vještina u *SearchComponenti*

DAN 27.
08.11.2024

Za današnji zadatak fokusirali smo se na **usavršavanje *AuthComponent***, nakon što smo ju na samom početku implementirali i postavili glavne funkcionalnosti danas smo se pozabavili uklanjanjem grešaka, dodavanjem dodatnih metoda koje

su nam potrebne, ili će nam u daljnjoj razradi našeg plana biti potrebne, te dodavanje dizajna komponente kako bi poboljšali korisničko iskustvo.

Metode koje smo dodali u *AuthService*, kako bi usavršili ovaj servis, su sljedeće:

- *saveUserData()*
- *uploadProfileImage()*
- *getCurrentUserDetails()*
- *checkEmailExists*

U datoteku komponente tj. *AuthComponent* dodajemo sljedeće metode:

- *matchValues()*
- *togglePasswordVisibility()*
- *onFileSelected()*

Dizajn komponente:

1. Transformacije forme:

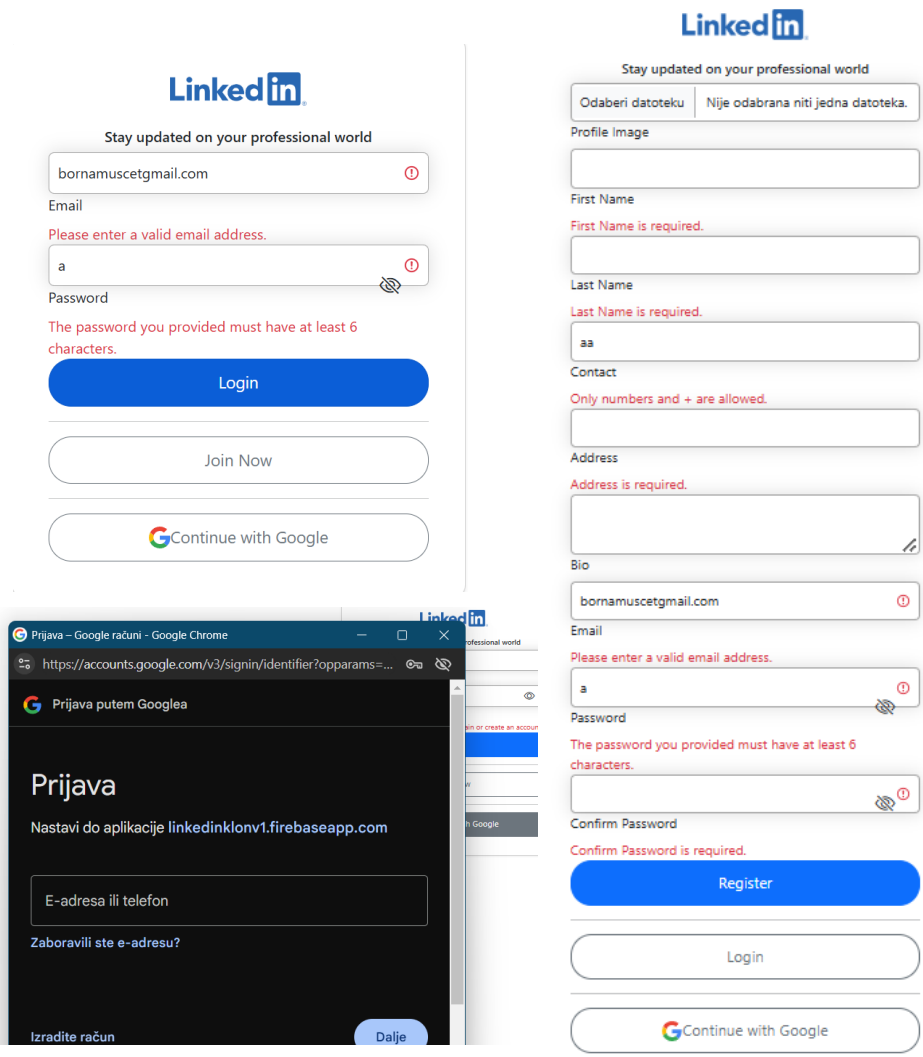
- Ograničavamo širinu forme kako bi postigli preglednost na širokim zaslonima, dok bi se na manjim forma prikazivala u centru
- Dodali smo i Media query koji za manje ekrane dodatno smanjuje padding forme, osigurava normalan izgled na svim uređajima

2. Prikaz logotipa LinkedIn-a

- dodali smo logotip LinkedIn-a, koji je centriran unutar forme.

3. Forme i validacije

- Dodali smo prikaz validacijskih grešaka ispod svakog input polja kada je aktivirano i označeno kao nevažeće, što poboljšava povratne informacije korisnicima

	 Prikaz različitih stanja AuthComponente
DAN 28. 19.11.2024.	Danas smo se ponovno vratili na <i>HeaderComponent</i>, s ciljem da dovršimo funkcionalnost za pretragu korisnika. U ovoj komponenti, funkcionalnost pretraživanja korisnika odvija se kroz search bar koji koristi metode definirane u <i>userService</i> i interakcijom s Firestore-om kako bi se dohvatili korisnički podaci koji odgovaraju pretrazi. Rezultati pretrage prikazuju se kao lista u padajućem izborniku a za svaki rezultat prikazana je slika, ime i prezime korisnika, klikom na rezultat poziva se metoda <i>viewProfile()</i> koja preusmjerava korisnika na profil odabranog rezultata. U <i>userService</i> koristimo <i>searchUsers()</i> metodu koja dohvaća sve korisničke dokumente u <i>users</i> kolekciji.

```

@Injectable({
  providedIn: 'root',
})
export class UserService {
  constructor(private firestore: AngularFireStore) {}

  searchUsers(query: string): Observable<any[]> {
    return this.firestore
      .collection('users')
      .valueChanges()
      .pipe(
        map((users) =>
          users.filter(
            (user: any) =>
              (user.firstName &&
                user.firstName.toLowerCase().includes(query.toLowerCase())) ||
              (user.lastName &&
                user.lastName.toLowerCase().includes(query.toLowerCase()))
            )
          )
      );
  }
}

```

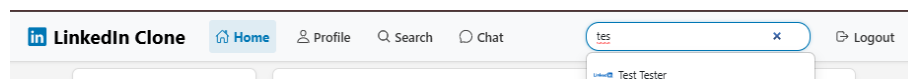
Prikaz metode za pretragu korisnika



Search bar u HeaderComponent

Ostatak dana bazirali smo se na **dizajna HeaderComponent**:

- cijela navigacijska traka postavljena je kao *sticky*, što znači da ostaje na vrhu tijekom listanja stranice.
- Dodali smo i *ion-icon* za LinkedIn logotip s plavom bojom
- Glavni navigacijski linkovi koriste isto tako *ion-icon* ikone
- Ulazno polje za pretraživanje smješteno je unutar *d-flex* forme za odmak od ostalih elemenata.
- Padajući meni prikazuje rezultate pretraživanja smješten ispod search bara, koristi sjene i zaobljene rubove.
- Koristi i *Bootstrap* klase poput *container-fluid* i *navbar-expand-lg*, što omogućuju prilagodbu navigacije ovisno o veličini ekrana.



Izgled HeaderComponent-e

DAN 29.
20.11.2024

Danas se nadovezujemo na zadatak od prethodnog dana, danas smo implementirali **dva različita tipa prikaza HeaderComponent**, tj. njenih dijelova. Naime u ovoj implementaciji korisničkog profila, razlikujemo dva prikaza:

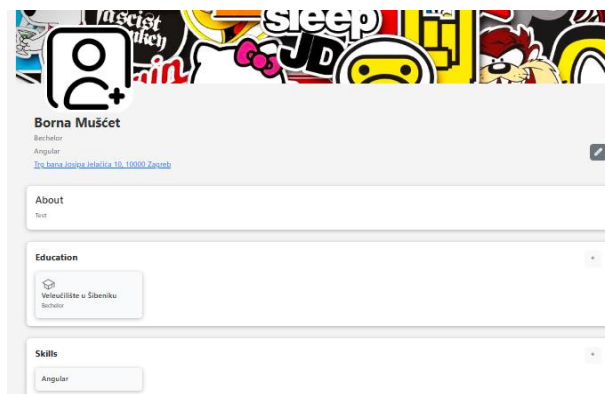
- Profil trenutno prijavljenog korisnika
- Profil drugih korisnika – u kojem nema mogućnosti uređivanja profila

Postizanje ovih razina ovisi o stanju *isCurrentUserProfile* varijable, koju smo postavili u *ProfileComponent*.

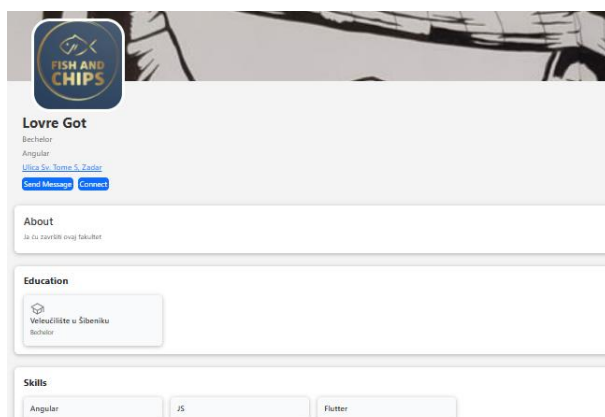
```
ngOnInit(): void {
  this.route.params.subscribe(async (params) => {
    this.userId = params['userId'];
    // Ako nema userId iz URL-a, postavi trenutnog korisnika
    if (!this.userId) {
      const currentUser = await this.afAuth.currentUser;
      if (currentUser) {
        this.userId = currentUser.uid;
        this.isCurrentUserProfile = true;
      }
    }

    if (this.userId) {
      await this.loadUserDataAndPosts();
      this.checkConnection();
    }
  });
}
```

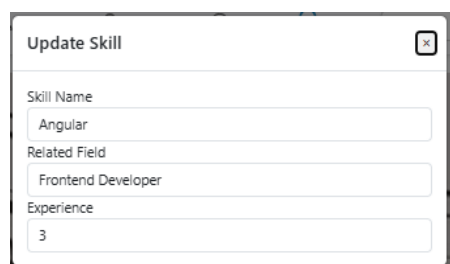
Utvrđivanje korisnika u URL-u



Prikaz profila trenutno prijavljenog korisnika



Prikaz profila drugog korisnika

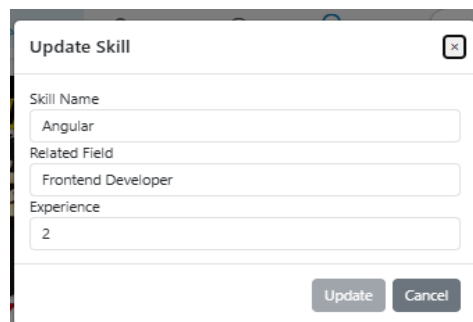


Update Skill

Skill Name
Angular

Related Field
Frontend Developer

Experience
3



Update Skill

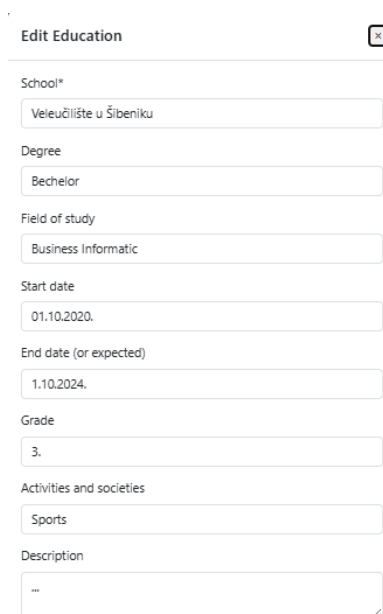
Skill Name
Angular

Related Field
Frontend Developer

Experience
2

Update Cancel

Razlike SkillsModala s obzirom na stanje korisnika



Edit Education

School*
Veleučilište u Šibeniku

Degree
Bachelor

Field of study
Business Informatic

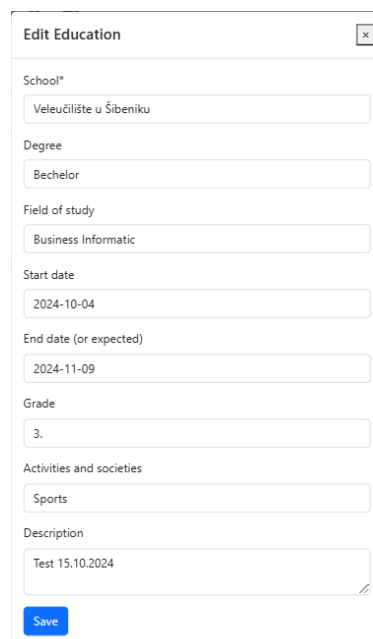
Start date
01.10.2020.

End date (or expected)
1.10.2024.

Grade
3.

Activities and societies
Sports

Description
...



Edit Education

School*
Veleučilište u Šibeniku

Degree
Bachelor

Field of study
Business Informatic

Start date
2024-10-04

End date (or expected)
2024-11-09

Grade
3.

Activities and societies
Sports

Description
Test 15.10.2024

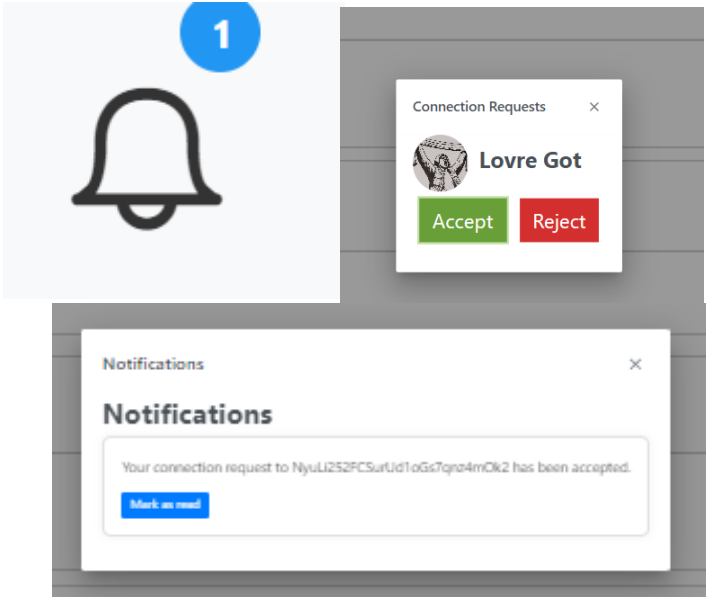
Save

Prikaz EducationModala s obzirom na stanje korisnika

Uz to na *ProfileComponent* dodali smo i dva gumba koja se prikazuju u trenutcima kada korisnik posjećuje profil drugog korisnika.

Gumb „Send Message“. Funkcija „startChat()“ usmjerava korisnika na stranicu za razgovor i prosljeđuje ID korisnika čiji se profil pregledava.

Gumb „Connect“ dodana je ali njegova funkcionalnost postaviti će se u daljnjoj razradi projekta.

	<pre> </p> <button class="btn btn-primary me-2" *ngIf="!isCurrentUserProfile" (click)="startChat()" > Send Message </button> </pre> <pre> startChat(): void { if (this.userId) { this.router.navigate(['/chat', { userId: this.userId }]); } } </pre> <i>Implementacija gumba „Send Message“</i>
DAN 30. 21.11.2024.	Sljedeći zadatak koji smo si postavili i zamislili bio je dodavanje funkcionalnosti slanja i primanja zahtjeva za povezivanje između korisnika i rukovanje s obavijestima. Ova implementacija omogućila je integraciju praćenja u stvarnom vremenu, prihvatanja, odbijanja zahtjeva te prikaz i upravljanje obavijestima. Ključne točke na koje smo obratili pažnju su: - Real-time ažuriranje - Upravljanje stanjem (<i>pending, accepted, rejected</i>) - Prilagodljiv prikaz zahtjeva i obavijesti u modalu U <i>HeaderComponent</i> dodali smo ikonicu zvona koja prikazuje broj nepročitanih obavijesti i zahtjeva. Zahtjevi se prikazuju kao lista s gumbovima za prihvatanje i odbijanje dok se obavijesti prikazuju kao lista s opcijom za označavanje kao pročitane.  <i>Prikaz ikonice i p-dialoga zaduženog za prikaz modala</i>

Nakon toga sljedeće što smo dodali bio je servis koji će biti zadužen za rukovanje sa obavijestima i zahtjevima korisnika, **NotificationService**. Neke od glavnih funkcionalnosti ovog servisa su sljedeće:

- Praćenje zahtjeva za povezivanje
- Dohvaćanje nepročitanih obavijesti
- Prihvatanje zahtjeva za povezivanje
- Odbijanje zahtjeva za povezivanje
- Označavanje obavijesti kao pročitane

```
// Zatim pratimo zahtjeve za povezivanje koristeći dobiveni us
this.notificationService
  .trackConnectionRequests(this.userId)
  .subscribe((requests) => {
    this.unreadRequests = requests;
    console.log('Unread Requests:', this.unreadRequests);
  });

this.loadUnreadRequests(user.uid);
this.loadUnreadNotifications(user.uid);
```

Prikaz dohvaćanja zahtjeva i obavijesti

```
// Funkcija za prihvatanje zahtjeva za povezivanje
acceptRequest(request: any) {
  this.notificationService.acceptConnectionRequest(request).then(() => {
    this.unreadRequests = this.unreadRequests.filter(
      (req) => req.id !== request.id
    );
  });
}

// Funkcija za odbijanje zahtjeva
rejectRequest(request: any) {
  this.notificationService.rejectConnectionRequest(request).then(() => {
    this.unreadRequests = this.unreadRequests.filter(
      (req) => req.id !== request.id
    );
  });
}
```

Prikaz metoda za prihvatanje i odbijanje zahtjeva

```
Full request data: notification.service.ts:23
{status: 'pending', to: '2ng5r6AVqrTzLyg1K2Zn39RUR6G3', timestamp: _Timestamp, from:
  'Nyul1252FCsurUd1o6s7qnz4mOk2'}
  from: 'Nyul1252FCsurUd1o6s7qnz4mOk2'
  status: 'pending'
  timestamp: _Timestamp
    nanoseconds: 40000000
    seconds: 1726314577
    [[Prototype]]: Object
    to: '2ng5r6AVqrTzLyg1K2Zn39RUR6G3'
    [[Prototype]]: Object
      constructor: f Object()
      hasOwnProperty: f hasOwnProperty()
      isPrototypeOf: f isPrototypeOf()
      propertyIsEnumerable: f propertyIsEnumerable()
      toLocaleString: f toLocaleString()
      toString: f ()
      valueOf: f valueOf()
      __defineGetter__: f __defineGetter__()
      __defineSetter__: f __defineSetter__()
      __lookupGetter__: f __lookupGetter__()
      __lookupSetter__: f __lookupSetter__()
      __proto__: (...)
      get __proto__: f __proto__()
      set __proto__: f __proto__()
Unread Requests: > [{}]
```

Prikaz podataka zahtjeva za povezivanje u konzoli

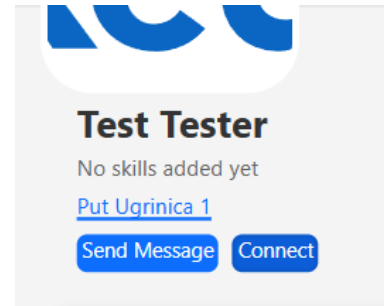
DAN 31.
22.11.2024.

Nastavili smo danas dalje s **razradom sustava za zahtjeve i obavijesti**, uz ranije kreiran gumb na *ProfileComponenti* dodali smo mu i funkcionalnost. Gumb

„Connect“ ima nekoliko svojstava koje vrijedi spomenuti, klikom na gumb šalje se zahtjev za povezivanje, naime gumb je vidljiv ako korisnik:

- pregledava profil drugog korisnika
- nije već ranije poslao zahtjev za povezivanje (!*connectionSent*)
- nije već povezan s korisnikom (!*isConnected*)

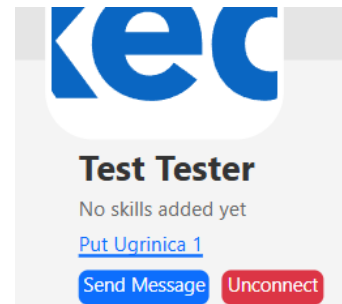
```
<button
  class="btn btn-primary"
  *ngIf="
    !isCurrentUserProfile && !connectionSent && !isConnected
  "
  (click)="sendConnectionRequest()"
  [attr.data-userid]="userId"
>
  Connect
</button>
```



prikaz gumba za slanje zahtjeva za povezivanje

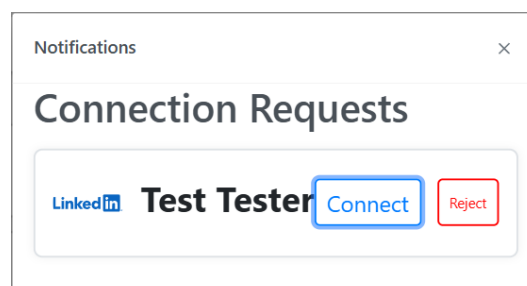
Gumb „Unconnect“ svojom aktivacijom, tj. klikom, pokreće proces prekida povezanosti između dva korisnika.

```
<button
  class="btn btn-outline-danger"
  *ngIf="!isCurrentUserProfile && isConnected"
  (click)="confirmUnconnect()"
>
  Unconnect
</button>
```



Prikaz gumba za prekid povezanosti dva korisnika

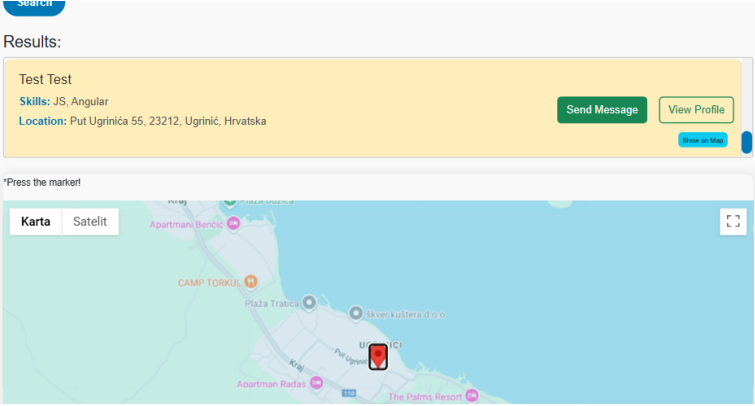
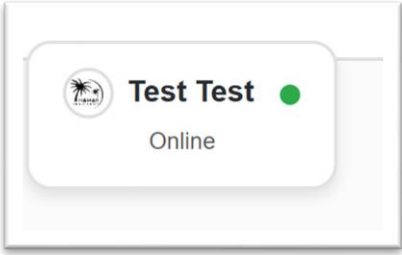
Nakon što smo implementirali servis i sustav za obavijesti i zahtjeve, trebali smo ga malo i urediti, stoga smo se pozabavili s css-om.

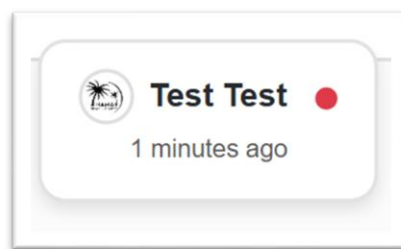


Prikaz zahtjeva za povezivanje

	Notifications Notifications Your connection request to Test Tester has been accepted. Mark as read Prikaz pristigle obavijesti
DAN 32. 26.11.2024.	Nakon implementacije sustava za obavijesti, sljedeći zadatak bio je dodavanje popisa koji će prikazivati listu povezanih korisnika na danas smo na <i>HomeComponenti</i>. Ovaj popis omogućuje pregledavanje njihovih profila i gumb koji vodi na komponentu za tekstualni razgovor. Direktiva <i>Angular-a *ngFor</i> iterira kroz niz <i>connectedUsers</i> i prikazuje svakog povezanog korisnika. Dodana je i dizajn korisničkog popisa, stiliziran je svaki element kako bi vizualno odvojio korisnike unutar popisa. <pre> <div class="col-lg-3 col-md-4 col-12"> <div class="card"> <div class="card-body"> <h5>Connections</h5> <ul class="list-group"> <li *ngFor="let user of connectedUsers" class="list-group-item-con"> <div class="d-flex align-items-center"> <div class="ms-2 clickable" (click)="goToProfile(user.uid)"> <h5>{{ user.firstName }} {{ user.lastName }}</h5> <h6>Skills: {{ skill.skillName }}<span *ngIf="i < user.skills.length - 1">, </h6> </div> <div class="ms-auto"> <button class="btn btn-primary send-message-btn" (click)="sendMessage(user.uid)"> <i class="bi bi-chat"></i> </button> </div> </div> </div> </div> </pre> Prikaz strukture dijela za povezane korisnike Podaci o povezanim korisnicima dohvaćamo preko <i>connectionIds</i>. To su ID-ovi s kojima je trenutni korisnik povezan. Oni su pohranjeni u Firestore-u unutar korisničkih podataka u polju <i>connections</i>. <pre> // Prihvatanje zahtjeva za povezivanje acceptConnectionRequest(request: any): Promise<void> { const userToUpdate = this.firestore .collection('users') .doc(request.to) .update({ connections: firebase.firestore.FieldValue.arrayUnion(request.from) }); } </pre>

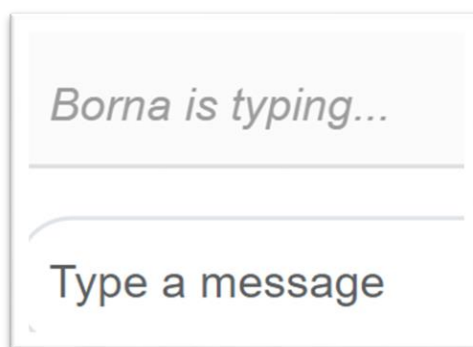
	Prikaz dohvaćanja podataka o povezanim korisnicima Connections  Test Tester Skills:   Test Test Skills: JS, Angular  Lista povezanih korisnika
DAN 33. 27.11.2024.	Danas smo se bazirali na nadogradnju SearchKomponente, u nju smo naime dodali par zanimljivih gumbova koji će nam omogućiti lakše i jednostavnije kretanje i izvršavanje potrebnih radnji u trenutku kada pretražujemo korisnike ovisno o kriterijima koje postavimo. HTML kod koji smo dodali prikazuje: - gumb za slanje zahtjeva za povezivanje ('Connect') - status gumba s obzirom na stanje zahtjeva ('pending') - gumb za slanje poruke na temelju trenutnog statusa veze između korisnika ('Send Message') - gumb za pregled profila korisnika ('View Profile') - gumb za centriranje određenog korisnika na mapi ('Show on Map') - Search Results: Lovre Got Skills: Angular, JS, Flutter Location: Ul. Ivana Mažuranića 7, 23000, Zadar, Hrvatska Connect View Profile Show on Map Search Results: Borna Miscet Skills: Ajajbshty Location: Put Ugrinića 55, 23212, Ugrinić, Hrvatska Pending View Profile Show on Map Search Results: Test Test Skills: JS, Angular Location: Put Ugrinića 55, 23212, Ugrinić, Hrvatska Send Message View Profile Show on Map

	<i>Prikaz dodanih gumbova i njihova različita stanja</i> Uz gumbove, dodali smo i mogućnost lakšeg pronalaska korisnika, naime klikom na marker koji se nalazi na karti pozivamo metodu <code>onMapMarkerClick()</code> koja omogućava isticanje korisnika na listi, koristi <code>scrollIntoView</code> za automatsko pomicanje do korisnika na listi te dodaje vizualni efekt ('<i>blink</i>') na odabranu stavku. Što znači, ako vidite marker na lokaciji 'Zagreb' klikom na njega u listi će vam se odmah pokazati taj korisnik na toj adresi.  <i>Prikaz korisnika koji se nalazi pod određenim markerom na karti</i>
DAN 34. 28.11.2024.	Zadatak današnjeg dana bio je usavršiti ChatComponentu, uz to što smo ju dodatno prilagodili korisniku, dodali smo joj i dizajn kako bi se uklopila u cijelu aplikaciju. Prva dodana stavka koju ćemo predstaviti biti će mogućnost praćenja statusa aktivnosti korisnika, tj. je li korisnik trenutno aktivan ili kada je bio zadnji puta aktivan. 



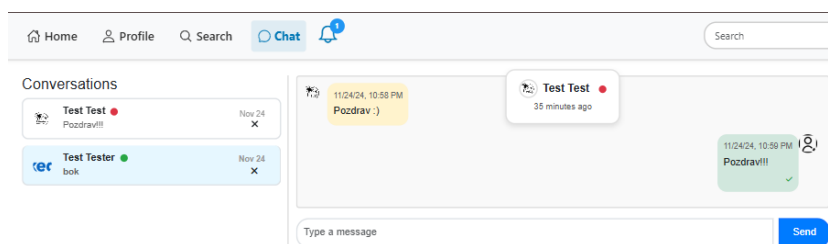
Prikaz statusa aktivnosti korisnika

Druga stavka koju smo dodali je mogućnost praćenja statusa tipkanja. Ovaj status omogućava prikaz poruke „Ivan is typing...” kada drugi korisnik tipka.



Prikaz statusa tipkanja drugog korisnika

Što se tiče dizajna komponente, dodali smo moderan stil koji komponenti povećava vizualni i funkcionalni dojam.



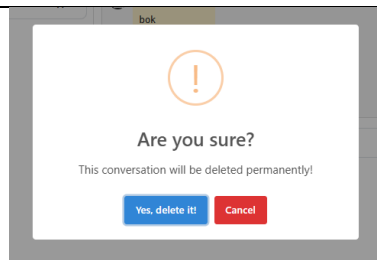
Prikaz ChatKomponente nakon usavršavanja

DAN 35.
29.11.2024.

Danas smo u naš projekt postavili nekoliko zanimljivih značajki kako bi dodatno poboljšali korisničko iskustvo. Dodali smo **biblioteke koje nam prikazuju određenu poruku obavijesti** ili nam otvara kreirane dijaloge koji nam koriste za potvrdu ili odustajanje od nekih akcija.

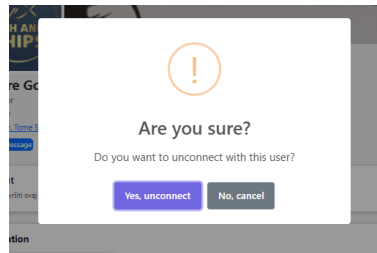
Prvo, *SweetAlert* – korišten je za prikazivanje dijaloga s upozorenjima i potvrdom korisničkih akcija, poput brisanja razgovora ili prekida povezanosti s korisnikom.

- Potvrda brisanja razgovora



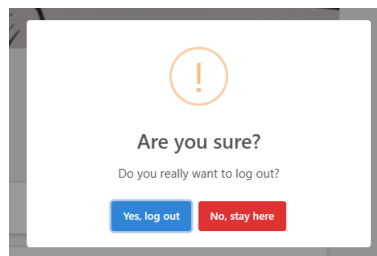
SweetAlert dijalog za brisanje razgovora

- Prekid povezanosti s korisnikom



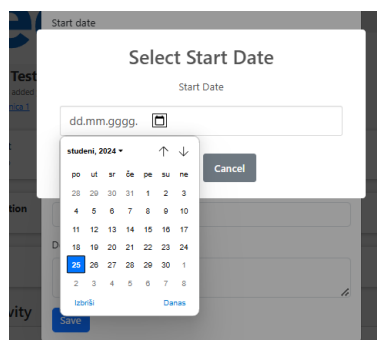
Dijalog za prekid povezanosti

- Odjavljivanje sa korisničkog profila



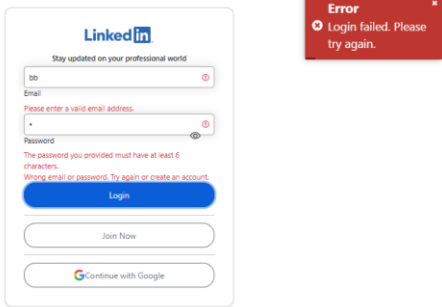
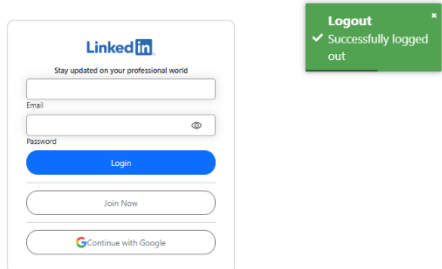
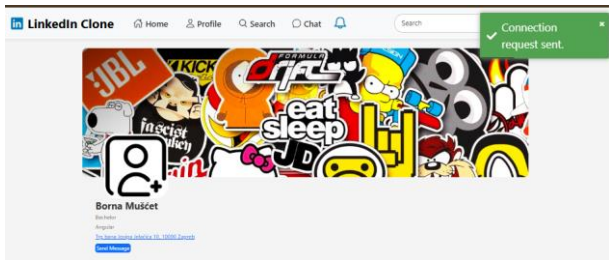
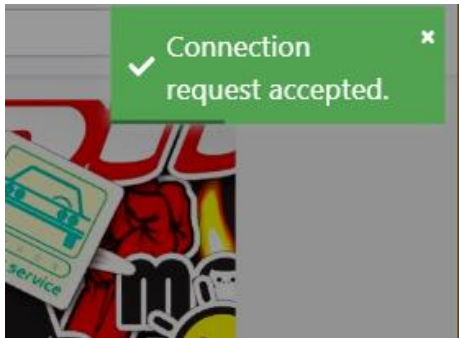
Dijalog odgovoran za odjavu korisnika

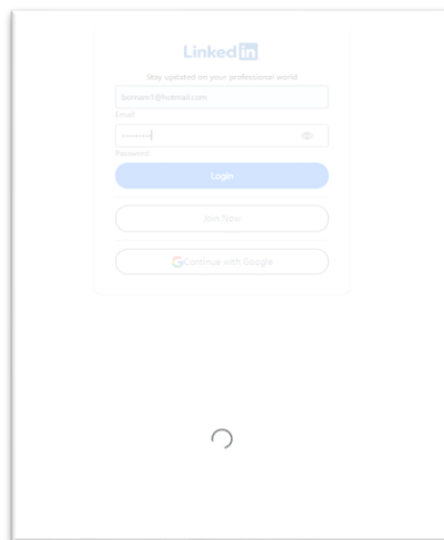
- Kalendar dodan u *EducationModalComponent*



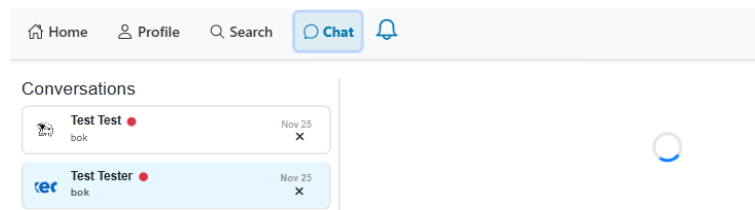
Dijalog za odabir datuma u EducationModalComponenti

Drugo, *Toastr* – knjižnica za prikaz brzih i nenametljivih obavijesti u web aplikacijama.

	 Prikaz Toastr 'error'  Prikaz Toastr 'success'  Obavijest za poslani zahtjev za povezivanje  Obavijest za prihvaćeni zahtjev za povezivanje
DAN 36. 02.12.2024.	Fokus današnjeg dana bio je na dodavanje varijable Loader koja koristi prilagođeni CSS za prikaz animacije i Angular funkcionalnosti za upravljanje vidljivošću tj. koristi se za označavanje da se neka radnja izvršava (učitavanje korisničkih podataka, slanje poruka itd.).



Prikaz Loader-a prilikom prijave



Prikaz Loadera u ChatComponenti

- Globalna kontrola u *AppComponenti* – ako postoji globalni *loading* status, može se prikazivati *loader* na cijeloj aplikaciji

Ostatak dana iskoristili smo za dodavanje funkcionalnosti koja upravlja neaktivnošću korisnika i osigurava njegovu odjavu ako je neaktivan određeno vrijeme, te postavlja korisnički status na '*offline*' prije izlaska iz aplikacije.

- Maksimalno vrijeme neaktivnosti korisnika u našem slučaju iznosi 15 minuta, nakon toga korisnik će automatski biti odjavljen.
- Koristimo *@HostListener* koje je zadužen za resetiranje brojila u slučaju da :
 - se pomakne miš
 - korisnik pritisne tipku na tipkovnici
 - korisnik klikne mišem
- postavili smo i obavijest u kojoj korisnik dobiva poruku putem *alert()* kako bi znao da je sesija istekla

```

ngOnInit(): void {
  this.resetTimer(); // Resetiraj timer prilikom inicijalizacije

  // Postavi interval za provjeru neaktivnosti
  this.timeout = setInterval(() => {
    this.idleTime += 1000; // Povećavaj vrijeme neaktivnosti svakih 1 sekundu
    if (this.idleTime >= this.maxIdleTime) {
      this.logoutUser();
    }
  }, 1000);

  // Resetiraj timer kada se dogodi aktivnost korisnika
  @HostListener('window:mousemove') onMouseMove() {
    this.resetTimer();
  }

  @HostListener('window:keydown') onKeyDown() {
    this.resetTimer();
  }

  @HostListener('window:click') onClick() {
    this.resetTimer();
  }

  @HostListener('window:beforeunload', ['$event'])
  unloadHandler(event: Event) {
    this.setUserOfflineBeforeUnload();
  }

  // Resetiranje timera za neaktivnost
  resetTimer() {
    this.idleTime = 0;
  }

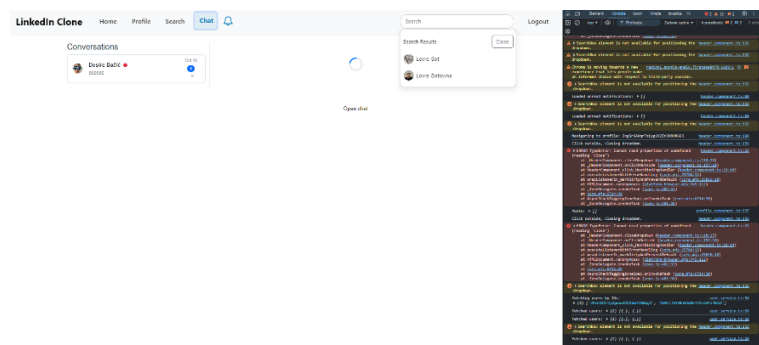
  // Funkcija za automatsku odjavu korisnika
  logoutUser() {
    clearInterval(this.timeout); // Očisti interval kada se korisnik odjavi
    this.authService.logout(); // Poziv funkcije za odjavu
    this.router.navigate(['/login']); // Preusmjeri na login stranicu
    alert('You have been logged out due to inactivity.');
```

Metode zadužene za automatsko odjavljivanje korisnika

DAN 37.
03.12.2024.

Skoro smo došli i do kraja rada na ovoj aplikaciji, današnji zadatak bazirali smo na **testiranju aplikacije** i svih njenih komponenti, gumbova i funkcionalnosti koje smo implementirali u nju. U tom procesu testiranja naišli smo na tri veća problema a to su:

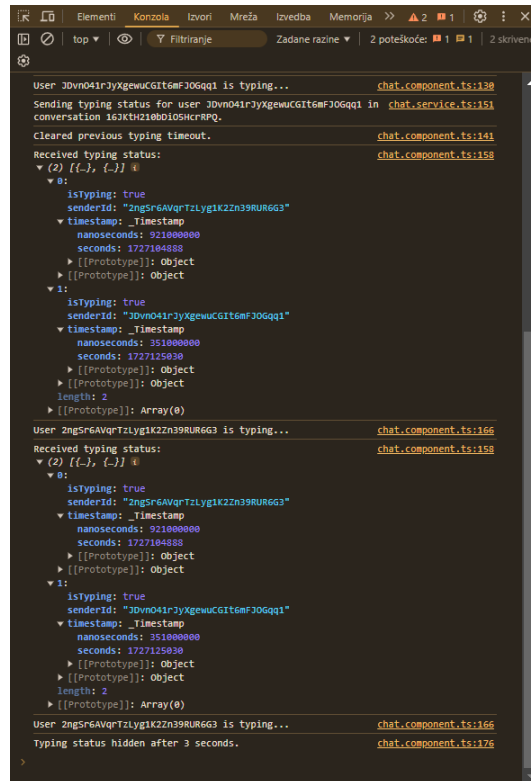
1. *Search bar* se nakon korištenja u trenutcima sam otvarao tj. njegov izbornik sa listom korisnika.



Pregled greške izbornika u navigacijskoj traci

2. Prikazivanje statusa tipkanja
 - Problem se pojavio tako da se status prikazivao za oba korisnika od jednom tj. status se prikazivao i korisniku koji piše i onom koji čeka

poruku. Status se prikazivao i prilikom otvaranja razgovora, iako nitko u trenutku nije tipkao.



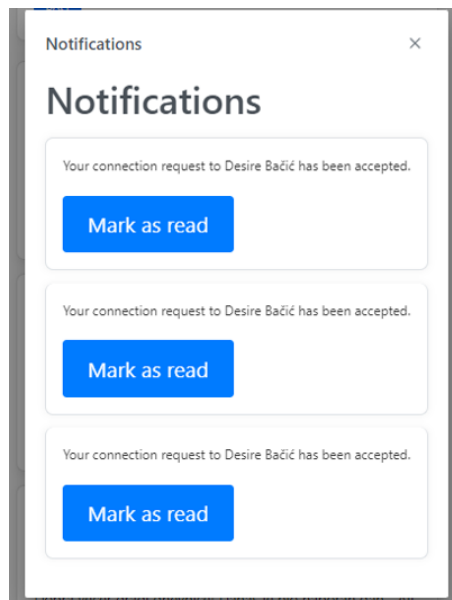
```

User 3Dvn041r3yXgewuGIt6mF30Gq1 is typing... chat.component.ts:138
Sending typing status for user 3Dvn041r3yXgewuGIt6mF30Gq1 in conversation 16JkH210D010SHcrAPQ. chat.service.ts:151
Cleared previous typing timeout. chat.component.ts:141
Received typing status: chat.component.ts:158
▼ (2) [{"-"}, {"-"}]
  0:
    isTyping: true
    senderId: "2ngSr6AVqTzlyg1K2Zn39RUR6G3"
    timestamp: _Timestamp
      nanoseconds: 921000000
      seconds: 1727104888
    [[Prototype]]: Object
  1:
    isTyping: true
    senderId: "3Dvn041r3yXgewuGIt6mF30Gq1"
    timestamp: _Timestamp
      nanoseconds: 351000000
      seconds: 1727125030
    [[Prototype]]: Object
  length: 2
  [[Prototype]]: Array(0)
User 2ngSr6AVqTzlyg1K2Zn39RUR6G3 is typing... chat.component.ts:166
Received typing status: chat.component.ts:158
▼ (2) [{"-"}, {"-"}]
  0:
    isTyping: true
    senderId: "2ngSr6AVqTzlyg1K2Zn39RUR6G3"
    timestamp: _Timestamp
      nanoseconds: 921000000
      seconds: 1727104888
    [[Prototype]]: Object
  1:
    isTyping: true
    senderId: "3Dvn041r3yXgewuGIt6mF30Gq1"
    timestamp: _Timestamp
      nanoseconds: 351000000
      seconds: 1727125030
    [[Prototype]]: Object
  length: 2
  [[Prototype]]: Array(0)
User 2ngSr6AVqTzlyg1K2Zn39RUR6G3 is typing... chat.component.ts:166
Typing status hidden after 3 seconds. chat.component.ts:176

```

Odgovori konzole u prilagođavanju statusa tipkanja

3. Prikazivanje iste obavijesti više puta



Greška višestrukog prikazivanje iste obavijesti .

DAN 37.
04.12.2024.

Današnji zadatak bio je **postavljanje repozitorija na GitHubu** za ovaj projekt. Uz kreiranje novog repozitorija na *GitHubu*, povezali smo lokalni projekt pomoću naredbi u terminalu.

Nakon toga, kako bi omogućili *hostanje* stranice putem *GitHub Pages*, morali smo:

- Konfigurirati projekt – koristili smo *Angular CLI* za generiranje *build* datoteka optimiziranih za produkciju, ovo je generiralo datoteke u mapi *dist*, potrebne za pokretanje aplikacije u pregledniku
- Nakon što smo *build* datoteke prenijeli na granu *gh-pages*, aktivirali smo *GitHub Pages* u postavkama, te smo generirali *URL* na kojoj se nalazi moja stranica

```
== Deploying to 'linkedinclove1'...
i deploying functions, hosting
i functions: preparing codebase default for deployment
i functions: ensuring required API cloudfunctions.googleapis.com is enabled...
i functions: ensuring required API cloudbuild.googleapis.com is enabled...
i artifactregistry: ensuring required API artifactregistry.googleapis.com is enabled...
+ functions: required API cloudfunctions.googleapis.com is enabled
+ functions: required API cloudbuild.googleapis.com is enabled
+ artifactregistry: required API artifactregistry.googleapis.com is enabled
! functions: package.json indicates an outdated version of firebase-functions. Please up
! functions: Please note that there will be breaking changes when you upgrade.
i functions: Loading and analyzing source code for codebase default to determine what to
! functions: You are using an old version of firebase-functions SDK (5.0.1). Please updat
i functions: preparing functions directory for uploading...
i functions: packaged C:\Angular\Angular Projekti\LinkedInCloneV1\functions (64.69 KB) fo
i hosting[linkedinclove1]: beginning deploy...
i hosting[linkedinclove1]: found 1 files in dist/LinkedInCloneV1
+ hosting[linkedinclove1]: file upload complete
i functions: Skipping the deploy of unchanged functions.
+ functions[addUserDatabase(us-central1)] Skipped (No changes detected)
+ functions: cleaning up build files...
! functions: Unhandled error cleaning up build images. This could result in a small month
i hosting[linkedinclove1]: finalizing version...
+ hosting[linkedinclove1]: version finalized
i hosting[linkedinclove1]: releasing new version...
+ hosting[linkedinclove1]: release complete

+ Deploy complete!

Project Console: https://console.firebase.google.com/project/linkedinclove1/overview
```

Prikaz radnji potrebnih za objavljivanje stranice na GitHub Pages

```
C:\Angular\Angular Projekti\LinkedInCloneV1>ng run build
i linkedinclove1: ng build
i ng build

Initial chunk files
Names      Raw size  Estimated transfer size
main-2148172.js      1.54 MB      275.86 KB
chunk-1738450.js     174.18 KB      42.12 KB
polyfills-d9a4d11.js  35.43 KB      11.75 KB
styles-89990c1.css    385 bytes      385 bytes
Initial total  1.75 MB      329.11 KB

Lazy chunk files
Names      Raw size  Estimated transfer size
main-123333000.css   28.49 KB      1.44 KB
chunk-102222101.js    23 bytes      23 bytes
Application bundle generation failed. [11.280 seconds]

✖ [ERROR] Polyfill for "angular/localize/init" was added automatically. [plugin angular-polyfills]

In the future, this functionality will be removed. Please add this polyfill in the "polyfills" section of your "angular.json" instead.

✖ [ERROR] bundle initial encoded maximum budget. Budget 512.00 KB was not met by 842.40 KB with a total of 1.35 MB.

✖ [ERROR] src/app/search/search.component.css exceeded maximum budget. Budget 2.40 KB was not met by 405 bytes with a total of 2.51 KB.

✖ [ERROR] src/app/header/header.component.css exceeded maximum budget. Budget 2.40 KB was not met by 1.49 KB with a total of 3.54 KB.

✖ [ERROR] src/app/profile/profile.component.css exceeded maximum budget. Budget 2.40 KB was not met by 1.70 KB with a total of 3.75 KB.

✖ [ERROR] src/app/chat/chat.component.css exceeded maximum budget. Budget 2.40 KB was not met by 1.01 KB with a total of 3.46 KB.

✖ [ERROR] src/app/home/home.component.css exceeded maximum budget. Budget 2.40 KB was not met by 1.27 KB with a total of 3.12 KB.

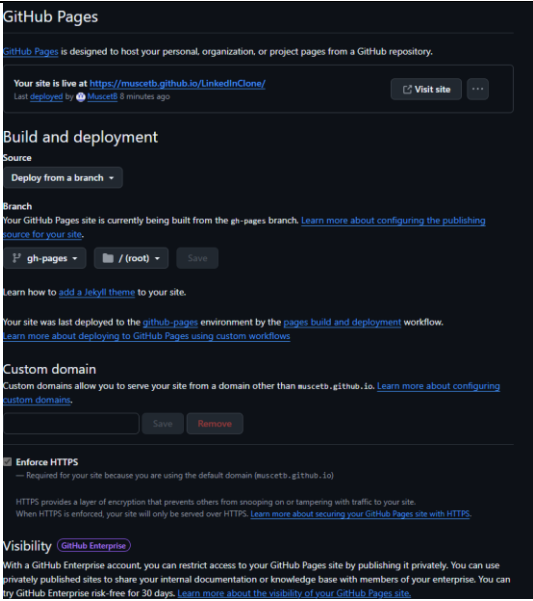
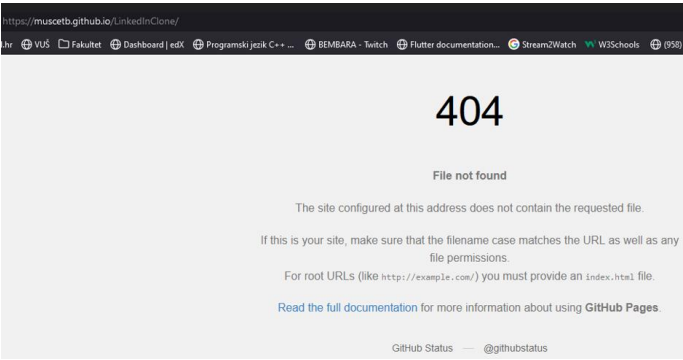
✖ [ERROR] Module "ua-parser-js" used by "src/app/chat/chat.component.ts" is not ESM

CommonJS or AMD dependencies can cause optimization bailouts.
For more information see: https://angular.dev/tools/cli/build#tree-shaking-commonjs-dependencies

✖ [ERROR] bundle initial encoded maximum budget. Budget 1.40 MB was not met by 305.82 KB with a total of 1.75 MB.

✖ [ERROR] C:\Angular\Angular Projekti\LinkedInCloneV1\dist\LinkedInCloneV1\main-2148172.js
```

Prikaz procesa build

	 <i>Prikaz postavki GitHub-a u kojem se nalazi URL</i>
DAN 38. 05.12.2024	Nažalost nakon jučerašnjeg dana URL na kojem je stranica objavljena nije bio u funkciji, klikom na URL prikazivala se 404 greška.  <i>Prikaz greške nastale na GitHub URL-u</i> Nakon dodatnog istraživanja na koji način pravilno odraditi sve potrebno za Angular <i>deploy</i> aplikacije na <i>GitHub Pages</i>, došli smo do rješenja: - Prvo što moramo popraviti, tj. dodati u <i>angular.json</i> datoteku je konfiguracija za Angular aplikaciju, moramo osigurati da je naša aplikacija konfigurirana za <i>deployment</i> u <i>root</i> direktorij.  <i>Prikaz konfiguracije angular.json datoteke</i>

- Umjesto obične *'npm run build'* direktive koristili smo sljedeću:
'ng build --output-path dist --base-href "/LinkedInClone/"

```
C:\Angular\Angular Projekti\LinkedInClone>ng build --output-path dist --base-href "/LinkedInClone/"
Initial chunk files | Names | Raw size | Estimated transfer size
main-SF3M6FSV.js | main | 1.15 MB | 275.46 kB
styles-BMGHQUU.css | styles | 624.00 kB | 71.31 kB
chunk-I7R3E4IA.js | - | 174.39 kB | 42.22 kB
scripts-ROIE3SDX.js | scripts | 95.34 kB | 25.91 kB
polyfills-Q8W4TBj.js | polyfills | 35.82 kB | 11.75 kB
| Initial total | 2.08 MB | 426.65 kB

Lazy chunk files | Names | Raw size | Estimated transfer size
main-IIO3AIVH.css | - | 26.05 kB | 1.44 kB
chunk-XXYSBYLN.js | index-esm | 29 bytes | 29 bytes

Application bundle generation complete. [11.816 seconds]

▲ [WARNING] Polyfill for "@angular/localize/init" was added automatically. [plugin angular-polyfills]

In the future, this functionality will be removed. Please add this polyfill in the "polyfills" sect

▲ [WARNING] 2 rules skipped due to selector errors:
.form-floating>~label -> Did not expect successive traversals.
.form-floating>~label -> Did not expect successive traversals.

Output location: C:\Angular\Angular Projekti\LinkedInClone\dist

C:\Angular\Angular Projekti\LinkedInClone>npm run angular-cli-ghpages --dir=dist
▲ [WARNING] 2 rules skipped due to selector errors:
.form-floating>~label -> Did not expect successive traversals.
.form-floating>~label -> Did not expect successive traversals.

Output location: C:\Angular\Angular Projekti\LinkedInClone\dist
```

Pregled nove build direktive

- Nakon *build-a* spremni smo za objavljivanje na *GitHub Pages*.

```
C:\Angular\Angular Projekti\LinkedInClone>npm run angular-cli-ghpages --dir=dist
index.html could not be copied to 404.html, Proceeding without it.
Diagnostic info: ENOENT: no such file or directory, lstat 'C:\Angular\Angular Projekti\LinkedInClone\dist\index.html'
.nojekyll file created
.nojekyll file created
🔥 Uploading via git, please wait...
Cloning https://github.com/MuscatB/LinkedInClone.git into C:\Angular\Angular Projekti\LinkedInClone\node_modules\cac
Cleaning
Fetching origin
Checking out origin/gh-pages
Removing files
Copying files
Adding all
Committing
Pushing
🌟 Successfully published via angular-cli-ghpages! Have a nice day!
```

Prikaz završne naredbe za objavljivanje

MuscatB Auto-generated commit 12 hours ago 57 Commits

assets/images

media

.nojekyll

3rdpartylicenses.txt

404.html

chunk-I7R3E4IA.js

chunk-XXYSBYLN.js

favicon.ico

index.html

main-IIO3AIVH.css

main-SF3M6FSV.js

polyfills-Q8W4TBj.js

scripts-ROIE3SDX.js

styles-BMGHQUU.css

Activity

0 stars

1 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Deployments 81

github-pages 12 hours ago

+ 80 deployments

Languages

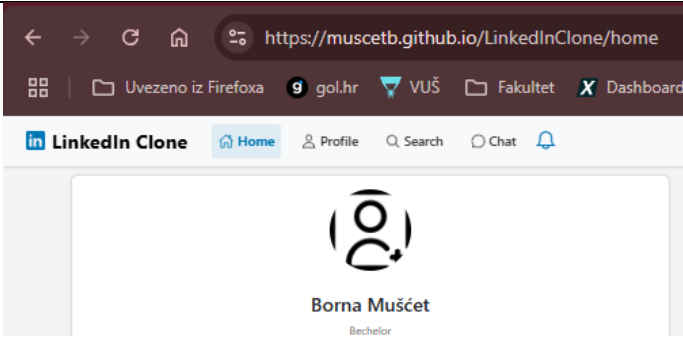
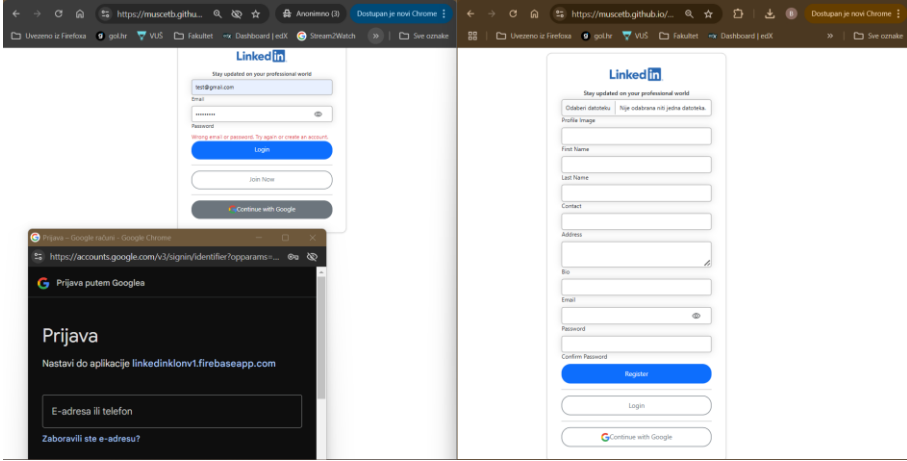
TypeScript 57.9%

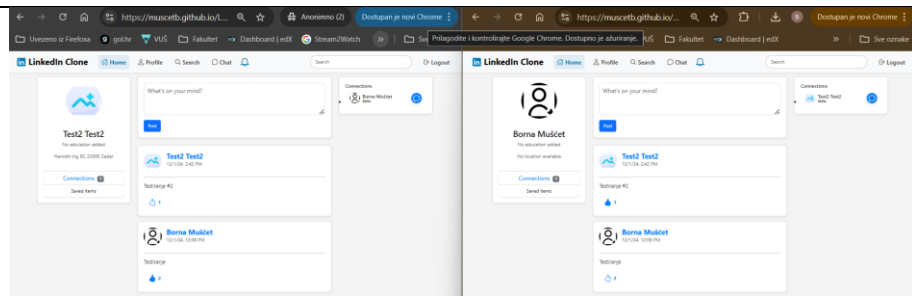
HTML 24.3%

CSS 17.6%

JavaScript 1.2%

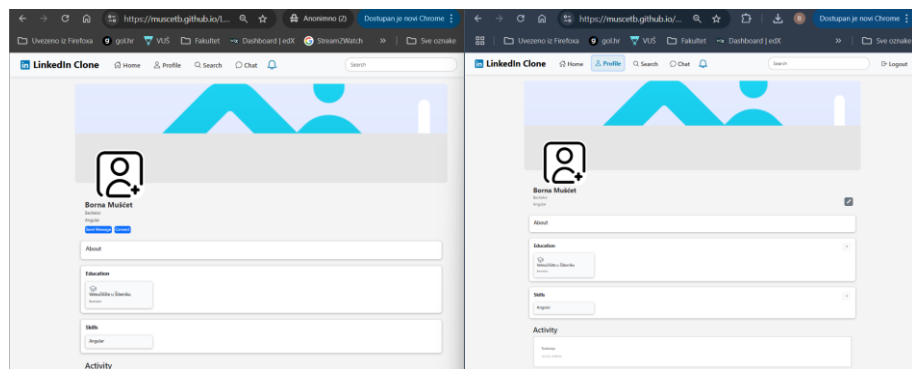
Prikaz direktorija LinkedInClone grane gh-pages nakon prijenosa

	 <i>Aplikacija se nalazi na URL-u GitHub Pages</i>
DAN 39. 06.12.2024	Tridesetdevetog dana praktične nastave završili smo naš projekt, sada kada je projekt završen i odjavljen na GitHub Pages URL-u, testirali smo cijelu aplikaciju s obzirom da se sada više ne nalazi na lokalnom direktoriju već na udaljenom. Jedan od problema koji se javio bio je problem sa <i>Google Sign-In</i> funkcionalnošću. Naime, prilikom pokušaja prijave preko Google Sing-In korisnik dobiva grešku vezanu za nedozvoljeni redirect URL. Uzrok tome je taj što <i>Google Sing In</i> zahtjeva da se <i>Authorized Redirect URLs</i> postavi u <i>Google Cloud Console</i> na točan URL stranice. Nakon toga fokusirali smo se na završno testiranje aplikacije i svih njenih funkcija na URL-u. - <i>AuthComponent</i>  <i>Prikaz AuthComponent</i> - <i>HomeComponent</i>



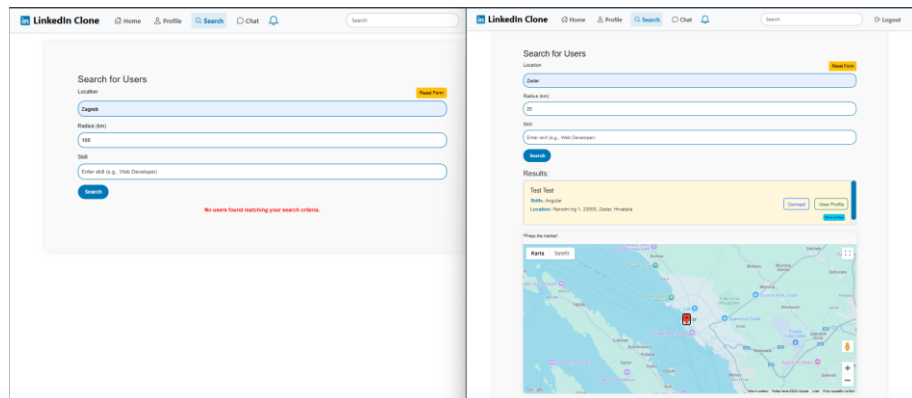
Prikaz HomeComponent

- ProfileComponent



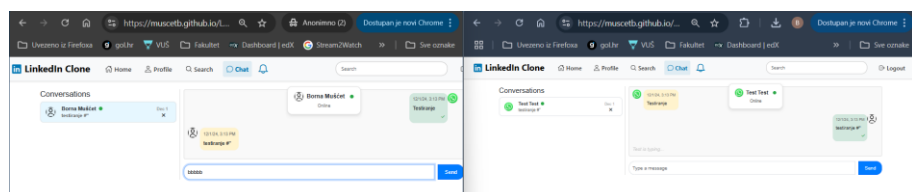
Prikaz ProfileComponent

- SearchComponent



Prikaz SearchComponent

- ChatComponent



	<i>Prikaz ChatKomponente</i> Nakon testiranja sve je spremno i projekt je uspješno završen te se nalazi na URL-u: https://muscetb.github.io/LinkedInClone/ https://github.com/MuscetB/LinkedInClone
ZAKLJUČAK OBAVLJENE STRUČNE PRAKSE	
Moje osobno mišljenje o stručnoj praksi izrazito je pozitivno. Tijekom ovog dijela obrazovanja imao sam priliku upoznati se s novim frameworkom za razvoj web aplikacija, Angularom, što mi je značajno proširilo tehnička znanja i vještine. Posebno bih istaknuo razvoj LinkedIn Clone aplikacije, koja je bila vrlo koristan i izazovan proces u praktičnoj primjeni stečenog znanja. Iako sam se susreo s nizom zahtjevnih izazova, iznimno sam zadovoljan postignutim rezultatima. Ovo iskustvo ne samo da mi je omogućilo bolje razumijevanje tehnologija i njihovih primjena, već me osnažilo i pripremio za buduće profesionalne izazove u IT-u. Što se tiče eventualnih mogućnosti za poboljšanje stručne prakse, nemam nikakvih primjedbi. Naprotiv, izuzetno sam zahvalan svom mentoru na pruženoj prilici, ideji, stručnom usmjeravanju i strpljenju. Također, mentoru dugujem veliku zahvalnost za omogućavanje pristupa edukacijskim materijalima, uključujući Udemy Business, što mi je pružilo dodatnu kvalitetnu edukaciju i trening uz koji sam usvojio Angular framework i unaprijedio svoje vještine. Sve u svemu, ovo iskustvo ocjenjujem kao vrlo korisno i nadahnjujuće, te vjerujem da će mi poslužiti kao čvrst temelj za daljnji profesionalni razvoj.	
POTPIS MENTORA STRUČNE PRAKSE  COGNISM d.o.o. Zadar OIB: 72843059121	MJESTO I DATUM: Zadar, 10.12.2024
	MJESTO I DATUM: Šibenik,